

IMPERIAL COLLEGE LONDON
DEPARTMENT OF AERONAUTICS

MENG GROUP DESIGN PROJECT 2019



Autonomous Drone Delivery

Technical Report - Flight Control

Student: Robert Atanasiu

CID: 01198290

1st Supervisor: Dr Ajit Panesar

2nd Supervisor: Dr Nigel MacCarthy

Date: June 17, 2019

Abstract

This paper aims to present an overview of the process of simulating an UAV's capability of completing its assigned mission profile. This mission was created by coupling multiple submissions characterized by requirements such as: GPS navigation, obstacle avoidance and target tracking. The main part of this report focuses on obstacle avoidance algorithms and their efficiency. D* Lite and a variation of Potential Fields have been implemented and the visual and numerical results given by simulations are presented in section *Simulations*. It was preferred to implement the Potential Fields algorithm in the final UAV configuration due to its lower computational necessities and successful tests conducted outdoors. The other part of this report presents the steps behind setting-up a virtual environment in *Gazebo Simulator* and how the physics engines have been designed. Section Integration of Algorithms presents an outlook on the integration process showcasing that the quadcopter is able to perform an integrated mission and not just isolated ones.

Acknowledgements

I would like to express my gratitude to my group members: Pablo Ferrer Bertomeu, Joel Leow, Lucian Nita, Jakob Stewart and Tian Wang. Their work ethic, innovative thinking and professional demeanor transformed this project into a very enjoyable learning experience. Thank you!

I would like to thank my supervisors, Dr Ajit Panesar, Dr Nigel MacCarthy and Dr Thulasi Mylvaganam for their advice and continuous involvement throughout these six weeks. Thank you for taking the time to explain difficult concepts and for teaching me what attention to detail really means. Your input was vital to this project and its success can be largely attributed to your guidance.

A special thanks goes to my group fellow, Gerald Low, for his help and effort provided in times of need.

Contents

Abstract	i
Acknowledgements	i
Table of Contents	ii
List of Figures	iv
List of Tables	iv
1 Introduction	1
1.1 Motivation	1
1.2 Overview	1
2 Literature Review	1
2.1 Potential Fields	1
2.1.1 Attractive Potential	2
2.1.2 Repulsive Potential	2
2.2 D* Lite	2
2.3 Proportional Navigation Tracking	3
3 Environmental Setup	3
3.1 Gazebo	3
3.2 ROS & MAVROS	4
4 Simulations	4
4.1 Physics Engines Plugins	5
4.1.1 Wind Field	5
4.1.2 Ground Effect	6
4.2 Mission Profiles	7
4.3 1 st Mission Profile	7
4.3.1 Results and Discussion	7
4.4 2 nd Mission Profile	8
4.4.1 Results and Discussion	9
4.5 General Observations	9
5 Integration of Algorithms	9
5.1 Improvements	10
5.1.1 Obstacle Avoidance & PN Tracking	10
6 Further Optimisation & Limitations	10
6.1 Hardware & Obstacle Avoidance	11
6.1.1 Local Minima Problem	11
6.1.2 D* Lite Lateral	11
6.2 Combined Implementation	11
6.3 Estimation of Wind Field	12
7 Conclusion	12
A Potential Fields Method Implementation	15
B ROS Nodes Topology with active Topics	16

C	Obstacle Avoidance Simulations	17
C.1	Unit Box Test	17
C.2	Unit Cylinder Test	18
C.3	Unit Sphere Test	19
C.4	Unit Combined Test	20
D	CPU & Memory Usage Comparison	21
E	Numerical Comparison Methodology	22
F	Yaw Rate PD Controller	23
G	Obstacle Avoidance & PN Tracking Simulations	24
H	Point Cloud & RViz	25
I	Dynamic Internal Agents States Solution	26
J	D* Lite Lateral Simulations	27
K	Wind Field Estimation Equation	28

List of Figures

2	Repulsive Potentials	2
3	Path Trajectory	2
4	IRIS Model quadcopter	3
5	Final IRIS Model Configuration	4
6	ROS Nodes Topology Computational Graph GUI	4
7	Gazebo representation of the wind field	5
8	General Configuration of Wind Field	6
9	Power vs Height in Indoor Hover Regime	6
10	1 st Mission Profile	7
11	1 st Mission Side View Comparison	7
12	1 st Mission Top View Comparison	7
13	2 nd Mission Configuration	8
14	2 nd Mission Side View Comparison	8
15	2 nd Mission Top View Comparison	9
16	Algorithm Integration Topology	10
17	Obstacle Avoidance and PN Tracking	10
18	Local Minima Problem	11
19	D* Lite Lateral	11
20	Full Mission ROS Node Topology	16
21	Unit box obstacle in Gazebo	17
22	Trajectory Over Unit Box	17
23	Simulated Wind Field	17
24	Unit Cylinder Obstacle in Gazebo	18
25	Trajectory Over Unit Cylinder	18
26	Simulated Wind Field	18
27	Unit Cylinder Obstacle in Gazebo	19
28	Trajectory Over Unit Sphere	19
29	Simulated Wind Field	19
30	Unit Geometries Obstacle Course in Gazebo	20
31	Trajectory Over Combined Unit Obstacles	20
32	Simulated Wind Field	20
33	Computational Performance Comparison	21
34	Height Trajectory Comparison	22
35	Lateral Trajectory Comparison	22
36	Yaw Comparison before/after PD Implementation	23
37	Trajectory Comparison	23
38	Orientation Comparison in <i>Gazebo</i>	23
39	1 st High-Complexity Integrated Mission Profile	24
40	2 nd High-Complexity Integrated Mission Profile	24
41	UAV identifying the obstacle using 3D Camera	25
42	Point Cloud Comparison in <i>RViz</i>	25
43	Solution for Local Minima Problem expressed in figure 18	26
44	1 st obstacle avoided; 2 nd obstacle crash	27
45	1 st obstacle crash; 2 nd obstacle crash	27
46	1 st obstacle crash then unexpected pathing	27

List of Tables

1	Comparison of Algorithms for 1 st Mission Profile	8
2	Comparison of Algorithms for 2 nd Mission Profile	9

1 Introduction

1.1 Motivation

The drone phenomenon has been continuously expanding during recent times and the social impact can be seen in rapidly increasing productivity levels across different industries: agriculture [1], military [2] and delivery services [3]. With constant technological advancements, the utility and adaptability of UAVs can be proved as beneficial in critical situations such as transport of medical goods. Currently, in demanding situations, this activity is performed by wheeled motor vehicles or manned aircraft, but these options can be deemed as costly and slow. The introduction of drones in this area could be an industry turning point, allowing for a faster and more efficient supply method in critical states. For example, the use of blood products including packed redblood cells (PRBCs), plasma and platelets has been shown to save lives in *bleeding trauma patients* [4]. Unfortunately, due to regulations and short shelf life (5 days for platelets), the supply present at a time in a hospital is limited. In terms of feasibility, the industry in Africa can be looked at as a certitude statement. Not only has Zipline introduced this type of drones in Rwanda, but it has also integrated them as a fully-functioning part in the blood-supply infrastructure [5]. Henceforth, the introduction of autonomously delivery drones would ensure a smoother and faster transportation network between hospitals and ambulances, increasing the probability of saving the injured victims while also creating an opportunity for more advanced missions, such as organ transportations.

1.2 Overview

The aim of this project was to design, build and test a quadrotor capable of performing a mission involving: autonomously picking-up a package, integration of GPS navigation with obstacle avoidance and tracking and landing on target via the use of vision algorithms. The *Flight Control* group has identified 3 different working sections and has split tasks accordingly; the main tasks were as follow:

- Obstacle Avoidance and target tracking;
- Hardware and Software integration;
- *Gazebo/Simulink* Simulations;

This report will focus on the results obtained in *Gazebo* regarding obstacle avoidance and compare the behaviour of the quadcopter in real-life and in the virtually created environmental setup. Furthermore, the author of this report opted to implement

another obstacle avoidance algorithm in parallel: *Potential Fields Method*; this comes as a solution regarding the Hardware assigned at the beginning of the project, with the oDroid Computer-Board having too low computational power to sustain all the of algorithms needed to be implemented. The reasons behind this decision will be further discussed in the *Simulations Section* where the computational performance of both algorithms is compared.

In the same order of ideas, all algorithms have been simulated in Gazebo and tested individually in real life. But a bundle of isolated algorithms will not provide functionality to the drone and thus, integration was a vital part in this project. Due to it's already developed familiarity with the ROS/Gazebo Environment, the author of this report has decided to fit together the following pieces of code:

- takeoff and GPS navigation;
- obstacle avoidance
- ambulance(target) PN tracking

Lastly, it is worth mentioning the limitations that arose during the project.

On one hand, having 2 obstacle avoidance algorithms gave the option of choosing the better one, but the UAV had to be equipped in different sensor setup configurations. This meant that the geometrical/physical properties of the drone had to be changed from test to test, which reduced the time-efficiency of *simulations*.

On the other hand, due to the high complexity of the project and numerous Hardware tools utilised, there was a large cumulative uncertainty value. Numerical methods and a general trend of replicating real life recorded results have been used as solutions to model phenomenons such as: effect of 3D wind field, ground effect and sensor/camera noise.

2 Literature Review

This section will focus on presenting the overview of the obstacle avoidance algorithms: *D* Lite* and *Potential Fields*, as well as describing the main features of the proportional navigation tracking algorithm.

2.1 Potential Fields

This path-planning algorithm can be mainly characterised by efficiency and low computational power necessity [6]. Eponymously, the environment in

which the drone performs its designated mission, will be characterized by a potential field $U(x)$, while sensor information will be used to approximate the potential field gradient $\nabla U(x)$.

The calculations determining the trajectory revolve around the 2 main parameters: the *goal*, which is surrounded by an attractive field and the *obstacle*, which is represented by a repulsive field [7]. The reason why this setup is widely used in industry is due to its fast computation time, which is explained from a mathematical standpoint. Instead of discretizing the environment as a 3D mesh, vector fields are utilised, thus allowing a definition of the potential field across the entire free space; with every timestep the attractive/repulsive potentials are updated and the drone moves accordingly.

2.1.1 Attractive Potential

The methodology through which the quadcopter is commanded to the desired position has been firstly modelled by Khatib [8]. The attractive potential can be written as:

$$U_{att}(s) = \frac{1}{2}k_{att}|s - s_g|^2 \quad (1)$$

where s is the current position of the drone, s_g is the goal position and k is the attractive constant. Now, by substituting equation 1 into Newton's second law of motion, the required acceleration can be computed. This result is further integrated to obtain a velocity, which is sent to the drone through ROS commands.

$$V = \int_{t^n}^{t^{n+1}} \frac{k_{att}}{m}(s - s_g)dt \quad (2)$$

2.1.2 Repulsive Potential

According to Budiyo [6], for the repulsive force, Krogh [9] proposed a new model which delivered better results, optimizing the manner in which a robot behaves in vicinity of an obstacle. Combining the repulsive potential function $U_{rep}(s)$ with the gradient rule, the repulsive force field can be determined as:

$$F_{rep}(s) = \begin{cases} \eta \left(1 - \frac{k_{rep}^2}{\rho_0}\right) \frac{1}{2a\rho_0} \frac{\partial \rho^2}{\partial s}, & \text{if } d \leq \rho_0 \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

$$\text{with } \frac{\partial \rho^2}{\partial s} = 2\rho_0 \frac{\rho_g - \rho_0}{\Delta s} \quad (4)$$

where d is the difference between current position and goal, ρ_0 represents the limit distance at

which the drone's response to the repulsive potential changes. In the present case, the flight regime switches from an oblique climb to a vertical one.

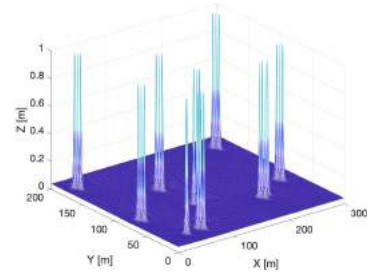


Figure 2: Repulsive Potentials

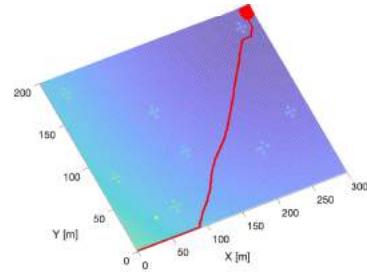


Figure 3: Path Trajectory

Following the pseudocode provided by Kala and Warwick [10], a Matlab script was implemented which provided the outcome shown in Figures 2 and 3. This methodology was then followed during the implementation of this algorithm on the drone via ROS, with a small change: the reference plane switched from X-Y to X-Z to further take advantage of reduced complexity and less sensor uncertainty. The detailed methodology is presented in Appendix A

2.2 D* Lite

The main idea behind D* Lite algorithm is a continuous replanning algorithm that is capable of computing the shortest path to the goal by minimizing one-step lookahead rhs values [11].

Its main requirement is to have an already predefined grid map, which would further allow a normal deployment of the mission. D* Lite combines not only combines specs of A^* and LPA^* , but rather it integrates them with predefined AI dynamic heuristic search method [12]. The results of this computational symbiosis is a very advanced path-planning algorithm with a very high success rate, that is able to provide the closest path to optimal at the moment.

The entire algorithm can be summed up with the following equation:

$$f(X, R) = g(X, R) + h(X) \quad (5)$$

where f is a function describing the attractiveness of neighbouring nodes, g is the expected distance to the goal position and h represent the *Euclidian* distance between current and goal positions. These positions are highlighted in equation 5 by states $\mathbf{X}$ and $\mathbf{R}$.

2.3 Proportional Navigation Tracking

The working principle of the PN algorithm is to keep the line of sight between the drone and the target constant, as this would eventually result in a collision. To enforce the opportunity of creating this "collision", the quadcopter's velocity vector has to rotate a variable amount at a constant timestep. This rotation will have to account for the target's change in direction observed in each individual time interval. [13].

The first step of this algorithm is to turn the drone in the direction of the target and set an initial velocity, which is further used to compute the acceleration vector. Next, after transforming all coordinates to a general global *FoR*[14], the relative positions are computed. The data is passed through a Kalman filter created by the *Sensors* group, converted into velocity inputs which are then sent to the drone.

Detecting that it is in vicinity of the target, the drone then switches to PID control using PX4 and in combination with the vision detection algorithm it perform a landing maneuver on the target.

3 Environmental Setup

3.1 Gazebo

The initial step in developing and testing the algorithms required to perform the designated mission, was to create a virtual world that has the ability of simulating real physics models. Gazebo is a 3D dynamic simulator whose interface allows advanced and efficient robot simulations in high-complexity environments [15].

At first, the open-source PX4 Firmware was downloaded and built in the catkin workspace. This package incorporates a visual default *iris* quadcopter model which was used throughout the simulations. The Firmware functions as a black-box, with the physics engines already incorporated to match the type of inputs Gazebo needs to perform the simulations. Thus, basic simple missions such as takeoff, hover and landing were available to be tested since the beginning.



Figure 4: IRIS Model quadcopter

But for more complex missions, sensors and cameras had to be introduced to detect obstacles and the AR tag positioned on the landing pad. Although *Gazebo* supports all the visuals required to perform a simulation, the physical world behind it is controlled by plugins written in the C++ ROS library. Hence, these tools had to be created/imported into the virtual world from 2 standpoints:

- physical/computational (introduction of real dynamic and kinematic properties)
- geometrical/visual

The first step came in the form of computational integration of TeraRanger/LiDAR sensors and RGB camera publishing directly to ROS topics using appropriate ROS wrappers. Then the data was processed to fit the requirements needed to implement all the previously described algorithms. One of the Hardware components that was assigned at the beginning of the project was the TeraRanger Tower, whose functionality is to detect and search for obstacles at 360°. Its initial configuration was comprised of 8×45° sensors, but it was considered redundant. Instead, it was opted for a 3×3° configuration, whose means were to only detect the orientation and the distance to the incoming obstacle. The integration was done by creating a TeraRanger node and cloning the package provided by Terabee on Github[16]. This allowed us to use all the real functionality of the sensors in *Gazebo* Simulation world. Thus, parameters such as *range to obstacle*, *maximum range* and *intensity* were stored and published to subscriber nodes which were just a few of the initialisers required in the constructor of our C++ created class, called *drone*. This class had the ultimate goal of determining the right set of commands which would create autonomous control throughout the mission.

The second step was represented by geometrical and visual integration of the mentioned visualization instruments through creation of .URDF and .SDF files; these type of files allowed the incorporation of physical properties such as: mass, moments of inertia, link and joint connections and sensor range and noise. It is important to mention that trying to emulate reality through virtual methods is hard and even though the means offered by *Gazebo* provided trustful results, approximations regarding noise and moments of inertia will influence the outcome of simulations. To model noise, a Gaussian probability density function has been applied to the field of view of sensors.

$$p_G(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}} \quad (6)$$

Thus, the parameters describing the purity of sensor readings are the standard deviation σ and mean μ , which come as an input to the user. Making use of the data collected by the *Sensor* group, it was decided that $\sigma = 0.2$ and $\mu = 0.1$ would best transition real effects to the virtual environment.

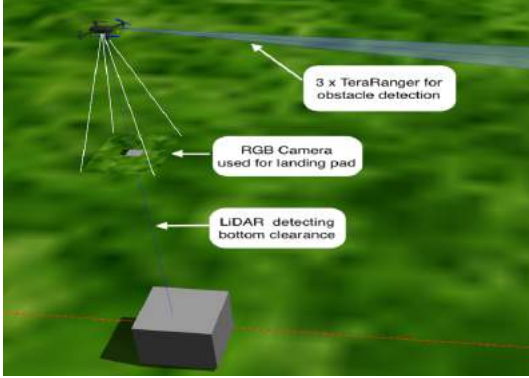


Figure 5: Final IRIS Model Configuration

3.2 ROS & MAVROS

ROS(*Robotics Operating System*) is an open-source Software which allows building and running code on any robot[17]. The development of the quadcopter presented in this project was done using the C++ version of the ROS Library. In the process of programming a drone, the most important ROS parameters are graph concepts which, in this case, come in the form of *nodes*, *messages* and *topics*.

Here, the nodes will publish a message to a topic, which will be then modelled, allowing the Offboard to send the right command to the drone. On the other hand, there needs to be a channel where ROS exchanges messages with the PX4 autopilot and this is the point where MAVROS and *MAVLink Communication* are introduced. The latter is a communication protocol between the PX4 autopilot and a ground station application. To summarize, MAVROS directly exchanges information with the autopilot through MAVLink Communication and publishes/subscribes to determined topics.

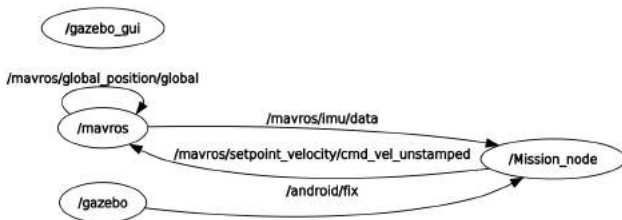


Figure 6: ROS Nodes Topology Computational Graph GUI

Figure 6 shows the interdependencies between the graph concepts. The *mavros* node contains the information published by the TeraRanger, LiDAR sensors and the RGB camera. When the master node *Mission_node* is started, the navigation algorithm is launched and with this, the class *drone* is initialized. Using object-oriented programming, a class of commands was built, allowing the users to send not only position inputs, but also velocity, acceleration and yaw commands. Appendix B presents the system architecture and how the links for the full integrated mission work.

In the backstage, the PX4 Firmware was communicating with *Gazebo* to receive sensor/camera data from the simulation world and send motor values to account for varying user commands. Then, the Offboard API(ROS) was communicating telemetry registered in the virtual world through MAVLink which was further used to simulate the physics engines offered by Gazebo GUI.

The last node constructed was the one designed to switch on/off the pump used to create suction in the gripper mechanism. Alongside, a pressure sensor was attached to the gripper and another publishing node was created; its only purpose was to continuously publish the force exerted by the mechanism on the package. Unfortunately due to the sensor's nature, the whole design process was done through Arduino and thus, ROS's nodes topology graph was not able to capture its links and topics.

4 Simulations

After the drone was equipped with all its sensors and the foundation of the virtual world was created, the only remaining requirement was to model the obstacles. Rendering high-complexity obstacles requires much more powerful computers and to adapt, a solution to decrease the CPU usage had to be found. Initially, a considered option was setting a lower barrier on the time step advancement rate by specifying a smaller updateRate tag in the world's .sdf physics block. Unfortunately, this update made the simulations very sluggish; and even though the subscribed sensors' publishing rate frequency was reduced to account for the slow *Gazebo* physics simulation, the mission deemed to be not realistic enough. Therefore, the best solution in this situation was to use only basic geometrical obstacles, such as cubes, spheres, cylinders and ramps.

4.1 Physics Engines Plugins

Testing at Silwood Park was a great opportunity to see the behaviour of the drone in an outdoor environment. In the greatest part it was found out that the integrated systems fit well together and the results are presented as a comparison with *Gazebo* ones in the *Simulations* section. But with this occasion, it was discovered that Gazebo's physics engines do not include all the relevant physical phenomena that could largely impact the behaviour of the drone during complex and intensive mission profiles such as obstacle avoidance.

To improve the overall structure of simulations it was decided that a couple of physics plugins had to be created. A plugin can be thought of as a piece of code which is compiled as a library, which afterwards is attributed to the world, model or sensor. This type of tool creates multitude of modelling opportunities, as it not only allows the user to create its desired physical engines, but also creates the possibility of modifying already integrated ones such as drag and lift considerations.

4.1.1 Wind Field

The implemented Potential Fields algorithm was first tested during the 3rd test at Silwood Park. Wind was a big factor at that time and the path followed by the drone was affected by it. When a gust of wind hit the quadcopter, a sideways momentum was created, which then translated into a shift in the UAV's lateral position. For a short period of time, the IMU destabilised and in order to re-gain equilibrium, it had to perform an automatic recalibration. After that, the drone could compensate for its miss-alignment and switch back to its original planned trajectory. Unfortunately the ROS library does not have an integrated plugin which emulates the effect of wind on a robot; a new one had to be created and 2 different methods of reproducing the disturbances created by wind have been analysed.

First, following the *RotorS* package designed by Furrer [18], a World Plugin which imposes an additional velocity to each of its components could be created. The first step would be the addition of a virtual wind sensor to the quadcopter; by subscribing to its only topic, which is the current airspeed, the drone would be able to compute the relative velocity and thus the disturbance created by this additional velocity. The problem in this case is that in reality, a wind sensor was not assigned at the beginning of this project, so a new option had

to be thought of.

Second, instead of imposing a velocity to the entire environment, a plugin that simulates the effect of a continuously moving force field has been designed and built. The field was shaped as a box using the properties of .sdf files and it was then programmed to follow the trajectory of the drone. This was done by creating an imaginary joint connection between the *iris* model and the middle of the box using geometrical and kinematic properties available in the declaration of .sdf files. The box was then discretised into a 3D mesh and increasing, but random forces have been applied in the cells corresponding to the local position of the quadcopter.



Figure 7: Gazebo representation of the wind field

Figure 7 shows how the wind field was visually interpreted by Gazebo. To show the manner in which the forces act, the 10 red balls have been added into the box, with the top one being fixed onto the vertical axis, whereas the last one would be always at least 10 cm above the drone. The other ones were jointed with their respective neighbours, but on none of them was there any condition imposed. These objects had no influence on the drone's flight behaviour and were created only to show the magnitude and direction of the wind. (up & right moves in example presented in figure 7)

To account for any possible position mismatch between the position of drone and the middle of the box, the forces have been extended to other cells in the box using extrapolation techniques. Because 3D extrapolation is not possible due to the nature of the surfaces created, it was decided to create slightly increasing X-dir. forces, while performing an extrapolation for the Y-Z plane. Furthermore, boundary conditions had to be enforced and it was set that no force is applied at any side of the box. The commands sent by the wind field, i.e forces imposed on the drone throughout the mission have been recorded using .bag files and then plotted in Matlab as shown by figure 8.

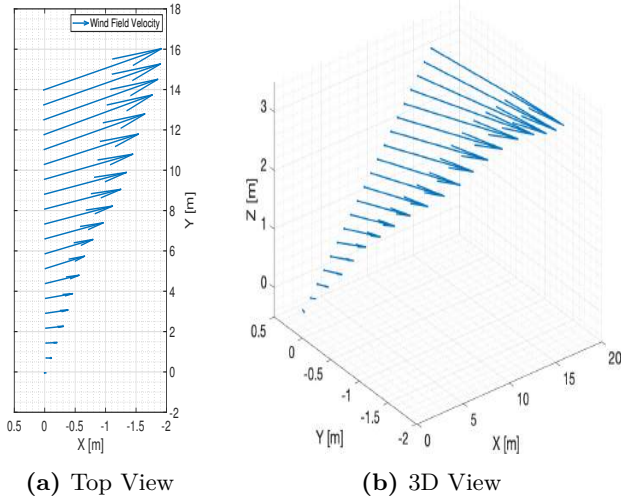


Figure 8: General Configuration of Wind Field

4.1.2 Ground Effect

The wake of an UAV can be largely affected when its rotors are close to obstacles that disturb its free development [19]. In most cases, the object that affects the normal behaviour of the drone is the ground surface. This phenomenon is mostly emphasized in hover and low speed flight regimes.

The main problem caused by ground effect is the UAV's stability which is affected by variations in drone's attitude (ϕ , θ , ψ). The angles largely depend on any additional imposed aerodynamics effects [20] and the impact can be summarized by an increase in lift and reduced propellers drag and angle of attack α . With these supplementary implications, the UAV becomes unstable for short time intervals. The onboard computer has to perform an automatic recalibration to counter the instability and to switch back to a general state of local equilibrium. This additional calculation largely increases the CPU and battery usage, thus decreasing the efficiency of the drone and the available flight-time. [21].

Following the manner in which the mission was split, it was observed that there were 3 phases in which *ground effect* might perturb the planned trajectory: when hovering and landing on the package; when going over obstacles and when approaching landing on the static target.

Before starting to present the approach to this problem it is worth mentioning that the precision of the LiDAR sensor used to compute height values was $\pm 25\text{cm}$. Thus, before going over an obstacle a condition on the flight behaviour of the drone was imposed. After reaching a sufficient altitude to perform a "go over obstacle" maneuver, the drone has to go up another 30 cm to ensure the safety

margin. Even though it is not optimal, the consequences of this decision decrease the probability of crashing into the obstacle. Based on approach and the CFD simulations presented in [22] it was decided that ground effect did not impact the flight regime at the transition over an obstacle.

Further on, an analysis showing the discrepancies regarding required *power input* between real conditions and *Gazebo* conditions has been created. Using the *Flight Arena* set up by the *Experimental Setup* group, 3 hover tests have been performed. Unfortunately the IMU and LiDAR sensor did not provide precise enough information required for an autonomous mission indoors, so the tests were performed manually with the aid of the safety pilot. The same mission, but with a different goal height was also simulated in *Gazebo*.

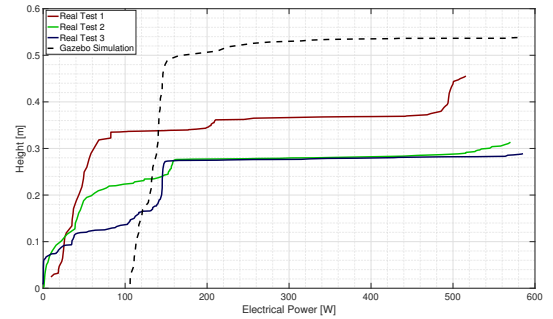


Figure 9: Power vs Height in Indoor Hover Regime

Figure 9 shows the variation of height with the input electrical power. The first observation is made with regards to very-low flight regimes [0-0.2m], where the required power in *Gazebo* is 10 times higher. This highly emphasizes the effect of ground effect which in reality pushes the quadcopter up. This effect will strongly affect the performance of the UAV when landing on the package as the maneuver has to be done slow and steady to ensure maximum precision. Going back to ROS physics plugins, it was decided to model this phenomenon using potential flow with 4 sources combined with method of images. The plugin will overwrite the *thrust* commands sent by ROS in low-height flight regimes [0-0.2m] and this value will vary according to the following formula [19]:

$$\frac{T_{GE}}{T} = \frac{1}{1 - \frac{R^2}{16z^2} - R^2 \left(\frac{z}{\sqrt{(d^2 + 4z^2)^3}} + \frac{z}{2\sqrt{(2d^2 + 4z^2)^3}} \right)}$$

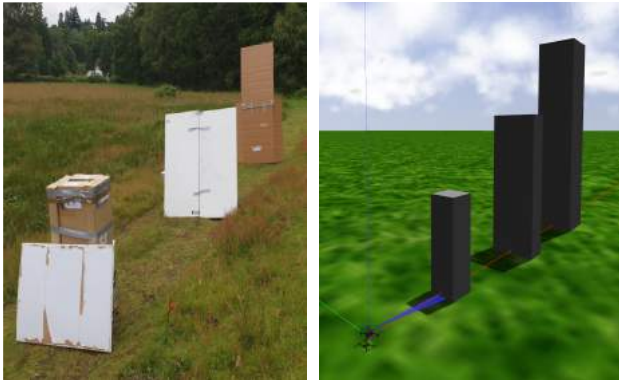
where T_{GE} is the new input thrust, R is the radius of each rotor, d is the separation distance between each 2 consecutive rotors and z is the height.

4.2 Mission Profiles

Having created the ideal virtual environment where physical engines simulate reality as close as possible, a number of missions profiles were designed to check the veracity of the implemented obstacle avoidance scripts. As mentioned previously, the obstacles were represented by basic geometrical shapes. This chapter will present the most important 2 missions profiles which incorporate all elements that can visually describe obstacles found in reality. For other simulations performed, please check Appendix C, where the evolution of simulations is presented.

- 1st mission will emulate the mission tested during the last session done at Silwood park; the obstacle course will comprise 3 rectangular cuboids of different heights.
- 2nd mission will comprise 3 different geometrical shapes; the main goal is to show that the drone was programmed to identify and avoid any type/combination of objects which is situated between its current position and the goal.

4.3 1st Mission Profile



(a) Silwood Obstacle Course (b) Gazebo Obstacle Course

Figure 10: 1st Mission Profile

Figure 10 presents the transition from reality to Gazebo. Some differences between the environments come in the form of ground inclination, as in the real test the obstacle course was mounted in a zone of $\approx 5^\circ$ inclination. To make the results even more reliable some basic trigonometrical manipulations have been applied to the data, ensuring that the ground is flat and set to 0m. It is important to mention that during the Silwood test the drone was equipped during 90% of tests with the variation of Potential Fields algorithm as this one provided better results and efficiency.

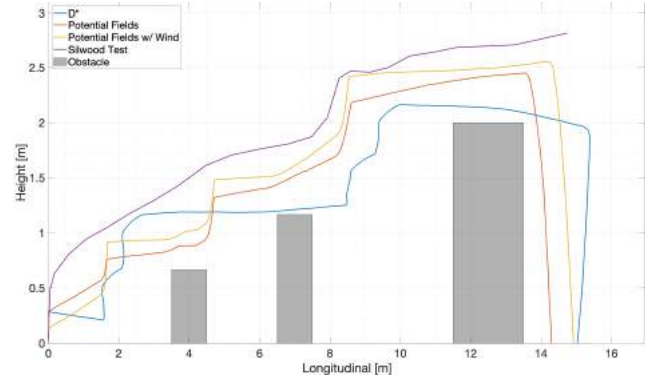


Figure 11: 1st Mission Side View Comparison

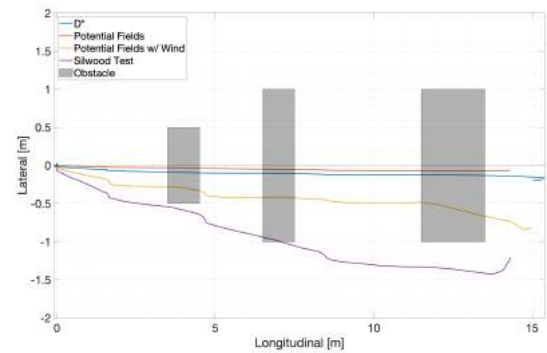


Figure 12: 1st Mission Top View Comparison

Figures 11 and 12 show a comparison between simulations done in *Gazebo* and their equivalent in reality. Even though the graphical interpretation of these plots showcases how each algorithm performed in comparison with the other ones, numerical evidence is required to show the efficiency and feasibility of each individual one.

4.3.1 Results and Discussion

Before presenting the results obtained in simulations it is worth explaining why the numbers obtained when using D* Lite at Silwood do not appear in figures 11 and 12. The oDroid computer board was not able to replan the trajectory more than a couple of times due to the sheer complexity of the algorithm. The memory got clogged, thus producing leaks leading to MAVROS timing out and the drone exhibiting unnatural behaviour. A plot presenting the difference regarding required computational power for both algorithms is presented in Appendix D.

Based on the previous affirmation, it was considered that the height values provided by Potential Fields method in *Gazebo* are closest to optimal ones. Thus, it was decided to compare the height values obtained with the other algorithms relative to the ones *Potential Filed* produced in *Gazebo*.

Regarding lateral trajectory, a slight offset can be observed in figure 12. Thus, it was decided that the reference line would be the straight one from origin, namely X-axis. Thus, the trajectories on Z-axis (figure 11) and Y-axis (figure 12) were compared, as well as the computational cost and running time required in each case.

Table 1: Comparison of Algorithms for 1st Mission Profile

Algorithm	σ_Y [m]	μ_Y [m]	σ_Z [m]	μ_Z [m]	Θ	t [s]
PF	0.01	0.01	/	/	9547	40.7
PF Wind	0.25	0.38	0.98	1.35	10282	43.6
D* Lite	0.02	0.02	1.37	-1.07	12432	53.1
Silwood	0.51	0.75	1.91	1.48	/	≈80

where PF stands for Potential Fields Methods, Θ for number of iterations and σ and μ for standard deviation, respective mean. The computational methodology is explained in Appendix E.

On one hand, figure 12 highlights the drone's tendency of having a lateral velocity component, even though there was no input sent from users specifying this condition. In reality, the strong wind conditions affected the UAV's ability to provide a precise state estimation, thus starting to move laterally to the right. This IMU decalibration problem escalated and the drone suffered from a continuous error in lateral direction propagating throughout the algorithm. In parallel, the simulated wind field produced similar results to the ones obtained at Silwood. Unfortunately, the uncertainty and error propagation created by the PX4 autopilot can not be modelled in *Gazebo* and the results are just a proof of concept. A subsection of *Further Improvements* presents a concept which can approximate the wind field magnitude and direction. Now, using the values provided by 1 it is concluded that D* and PF provided expected and similar results, deviating only by a very small amount from the reference axis.

On the other hand, figure 11 shows that the test ran at Silwood did not conclude with a landing maneuver, unlike the simulations. This is due to the *Experimental Setup* group interference who had to stop the test as a strong wind gust destabilised the drone even more. The drone correctly recorded the distance to the incoming obstacles and its pathing was appropriate, managing to avoid all the obstacles. The +25.0cm clearance distance between the UAV and the obstacle underneath it, was respected for all 3 cases.

Lastly, another test where the obstacle course would be formed by wider obstacles should be conducted in reality; in this way, the drone would be able to see and identify the last block for a larger time interval. Figure 11 showcases that the drone recognized the presence of the last obstacle only while being above the 2nd obstacle. After that, the UAV performed the remaining of its mission with the last recorded finite sensor readings. Therefore, the mission reached a sufficient level of success, the drone managing to clear all obstacles obstructing its path to the goal position. It has also shown that the created virtual environment is capable of creating reliable results, which have the ability of emulating real world conditions. Thus, all simulations done before this one had to be redone to analyse the effect of *Wind Field* and *Ground Effect*.

4.4 2nd Mission Profile

The 2nd mission profile was designed such that the algorithms are tested also against more complex geometries such as a sphere, a ramp and a cylinder.

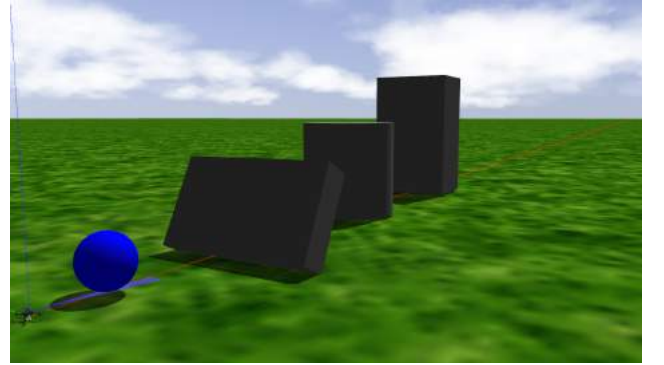


Figure 13: 2nd Mission Configuration

The UAV's completed the mission when using the 3 different algorithms, showcasing that the implementation of the renowned obstacle avoidance methods was successful.

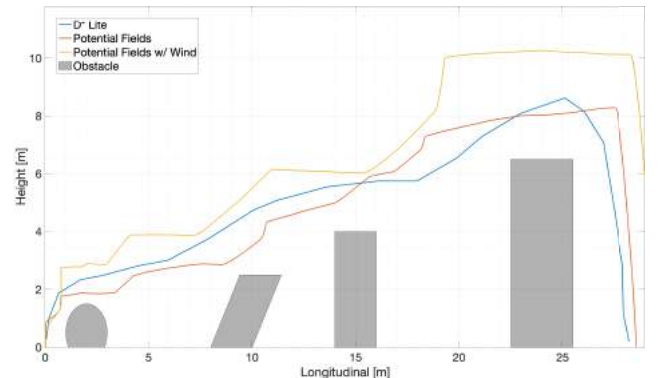


Figure 14: 2nd Mission Side View Comparison

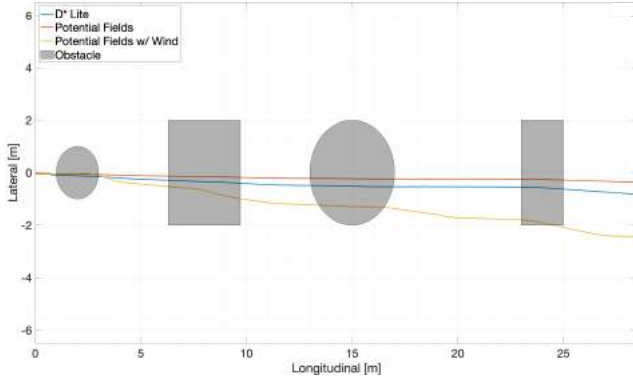


Figure 15: 2nd Mission Top View Comparison

4.4.1 Results and Discussion

Nonetheless, a numerical comparison had to be computed to highlight the efficiency of each algorithm. The 3 trajectories have also been compared as before, taking as references the Potential Field height trajectory and the X-axis.

Table 2: Comparison of Algorithms for 2nd Mission Profile

Algorithm	σ_Y [m]	μ_Y [m]	σ_Z [m]	μ_Z [m]	Θ	t [s]
PF	0.11	0.2	/	/	12857	99.6
PF Wind	0.75	0.92	0.79	1.22	14776	116.4
D* Lite	0.26	0.44	0.66	1.00	15840	125.7

Table 2 presents a numerical comparison recorded during the simulations for the 2nd mission profile. In retrospective, the results are similar to the ones presented for the 1st mission. It is worth mentioning that the Wind Field emulating Silwood conditions provided the expected output: the UAV shifted to the right, while flying at higher altitudes, thus overshooting.

4.5 General Observations

From both missions it can be observed that from a computational standpoint the *Potential Fields* method is more efficient than the *D* Lite*. In terms of CPU usage, represented in this case by number of iterations Θ , a relative difference of 26.7% can be perceived, while at the same time the running time of simulations is smaller by approximatively 28%.

From a trajectory standpoint, the *D* Lite* approaches the obstacles at a lower height as the +25.0cm restriction was not imposed due to the nature of the algorithm which is solely based on a created meshgrid which can not be altered. Concluding, both methods have been successful throughout simulations, but the *Potential Fields* has been chosen to be implemented in the final configuration

due to its higher efficiency and its already successful attempt in reality tested isolately at Silwood Park.

The last observation comes with regards to the *Gravity* plugin already incorporated in *Gazebo*. When forward motion inputs are sent, the drone has to create a pitching moment; it was noticed that the UAV has a tendency of going slowly, but steady downwards. Further investigation was conducted and it was found out that there is a *Gazebo* plugin which alters the gravity field (declared as $(0.0, 0.0, -9.801) \frac{m}{s^2}$ by $(0.0, 0.0, -0.1) \frac{m}{s^2}$ to account for various values recorded around Earth. This increases the pitching angle required and thus, to re-achieve stability the quadcopter has to change its attitude (ψ, θ, ϕ) . This phenomenon creates a small yaw ϕ component[23], whose effect can be seen both in figure 12 and 15 as there was no input from the user specifying that the UAV should move sideways when the wind plugin is not active.

A solution for this issue would be to delete the additional *Gravity* plugin and modify the values in the original script to the ones computed for the *London* area: $(0.0, 0.0, 9.816) \frac{m}{s^2}$ [24]. Unfortunately, the ROS/*Gazebo* Environment is a very complex one and deleting the additional plugin was not possible due to the large amount of interdependencies between their components. When one plugin is missing from a group of linked scripts, ROS will display an error and *Gazebo* will simply not launch.

5 Integration of Algorithms

Until this point in time, all simulations and real-life test have been done with isolated algorithms. But the sole purpose of this project is to create an UAV that can complete the required mission, thus the multitude of algorithms must be put together in a compatible form. Besides being compatible with each other, it has to be checked that the scripts do not interfere with each other at random times, i.e *vision tracking* does not start when the *obstacle avoidance* algorithm is running.

Figure 16 presents the order in which the algorithms will run during the mission profile. Even though a computational symbiosis that would allow parallel running of obstacle avoidance and PN tracking would be ideal, the specification of the oDroid did not allow the implementation of such a delicate design. With further improvements of hardware and computational iterations, the algorithms would be allowed to intertwine with each

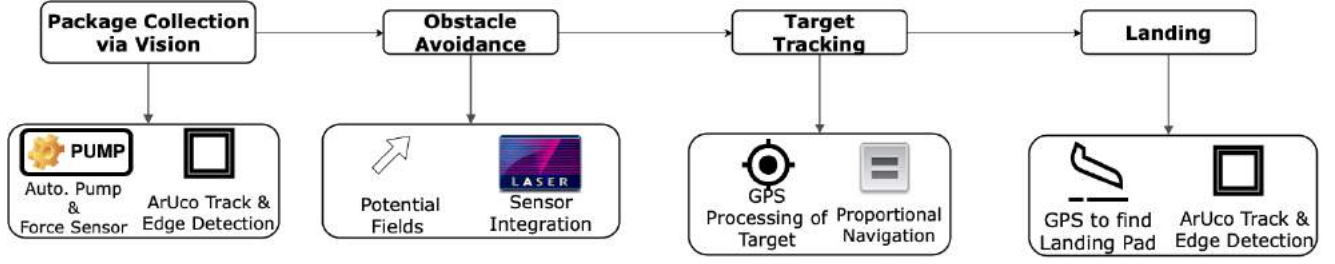


Figure 16: Algorithm Integration Topology

other, thus ensuring a more fluent deployment of the mission, from both a computational and energy-saving standpoint.

5.1 Improvements

During the implementation phase, the author of this project remarked that some bits of the provided source codes did not match well together. The observations were made in *Gazebo* and thus, improvements had to be created to ensure a smooth functioning of the scripts assembled together.

5.1.1 Obstacle Avoidance & PN Tracking

After clearing the obstacle course the drone received the GPS coordinates of the target and the initial command sent was to turn such that it faced it. After that, the PN algorithm was launched and the drone started to chase the ambulance. The problem in this situation was related to the heading angle, which was not the appropriate one, but rather a much smaller one, thus making the drone head in the wrong direction. This issue was caused by the large change in yaw required initially in a very short timestep. A PD controller for yaw-rate $\dot{\phi}$, such as the one described by [25] and [26], was enforced. PD controllers have the ultimate goal of increasing the stability of the system, since they have the ability of predicting the future error in system response [27].

$$\dot{\phi}(t) = K_p e(t) + K_d \frac{de(t)}{dt} \quad (7)$$

The attitude (ψ, θ, ϕ) of the drone was corrected and this can be seen in the *Gazebo* images from Appendix F. A yaw and trajectory comparison is also presented. It can be observed that after the implementation of the PD controller the drone managed to trace its target faster.

Having completed this task, the virtual *iris* quadcopter model was equipped with the script containing the integration version of *Potential Fields* and *PN Tracking*. The obstacle course was the same

one as the one presented in subsection *1st Mission Profile*.

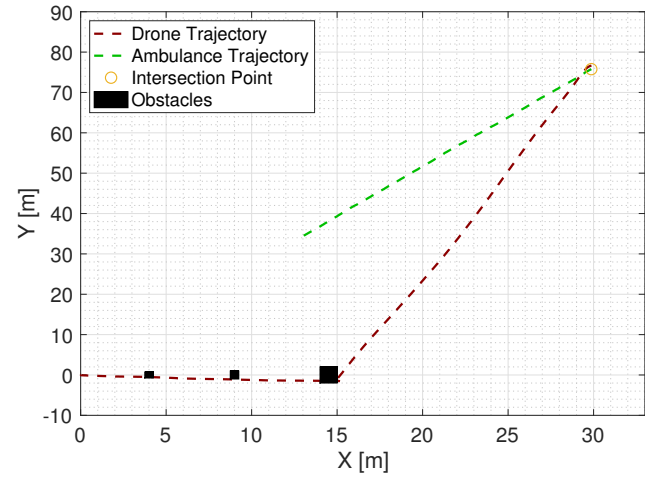


Figure 17: Obstacle Avoidance and PN Tracking

The integration method provided to be successful and this can be seen with the aid of figure 17. Importing the coordinates for both the UAV and ambulance from *Gazebo*, figure 17 was created. Completeness of mission was achieved as the drone managed to avoid the obstacles, then rotate to face the target and further track it. Even better, the UAV managed to catch the ambulance from behind before it stopped and then followed its path. This example showcases the high-efficiency of this algorithm. For further documentation, other mission profiles have been simulated and their results can be seen in Appendix G.

6 Further Optimisation & Limitations

With such short time constraints, the design process of the drone was in a large extent successful and the tests conducted at Silwood provided insightful results which can be used in the future to further optimize the core model.

6.1 Hardware & Obstacle Avoidance

At the beginning of the project, the *Flight Control* group decided that it would be best to implement a vertical obstacle avoidance algorithm and not a horizontal one. The limitation in this case comes in the UAV's inability of detecting obstacles above it. Using the methodology presented in the literature review for *Potential Fields*, the script can be extended to perform lateral avoidance as well.

But to achieve this, the hardware used has to be upgraded. The TeraRanger tower would be fitted on top of the UAV to perform 360° searches, while the 3D camera would be mounted upfront. The readings from the 3D camera would be transformed into a Point Cloud[28] and further into a depth map. The effects of these has been simulated in *RViz* and can be visualized in Appendix H. Through this method, the quadcopter would be able to perform a realistic localisation of obstacles, thus decreasing the need of continuously replanning caused by the uncertainty of obstacle position.

Now, with the transition to this method further theoretical concepts such as roadmaps [29] and visibility graphs [30] have to be introduced. The combinatorial planning techniques have been discussed in detail by my colleague from the *Flight Control* group and for further documentation please consult [31].

6.1.1 Local Minima Problem

The limitation in this case comes in the form of *Local Minima Problem*. The entire path-planning algorithm revolves around minimizing the cost(distance) and in order to do so, backpropagation using gradient descent is employed to find local minima positions.

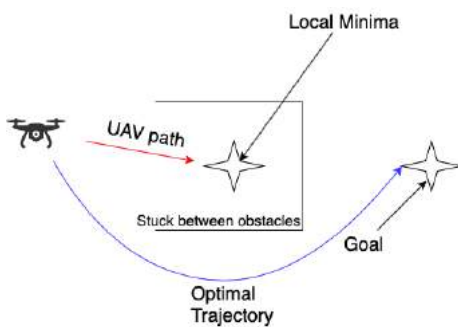


Figure 18: Local Minima Problem

Unfortunately, local minima positions are not always globally defined as minimum points and thus, the UAV might get stuck between obstacles. This

situation usually appears when the obstacle is represented by an open symmetrical section, where the drone is unable to decipher that it might get stuck in the section.

As it can be seen from fig 18 the UAV computes the shortest path, but not the optimal one. Following the methodology described by Mabrouk [32], this issue can be solved by employing dynamic internal agents states. The internal states are modeled as a dynamical system of coupled 1st ODEs that manipulate the potential field in which the agent is positioned. Equilibrium points are then manipulated by these internal states such that they become unstable equilibrium points, thus allowing the UAV to escape the local minima in the potential field. Further detail in I.

6.1.2 D* Lite Lateral

D* Lite is a multi-purpose algorithm and thus, the process of switching from X-Z plane to X-Y plane was very rapid. For further improvements, this was a starting point and *Gazebo* simulations provided insightful information.

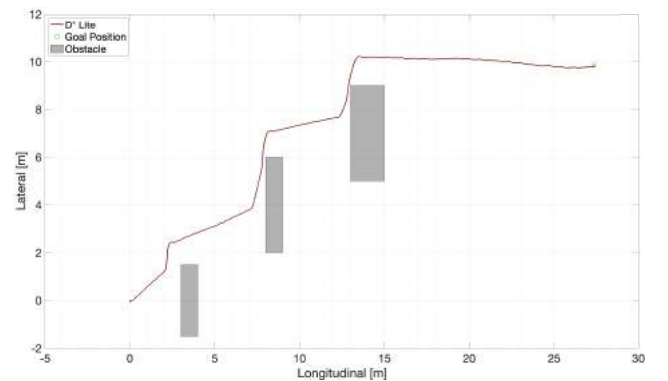


Figure 19: D* Lite Lateral

Although the UAV managed to complete a large majority of obstacle courses, it was found out that the algorithm is applicable only when the geometry of the obstacle is linear i.e box/ramp. The drone is not able to replan properly when more complex geometries are enforced, i.e sphere/cylinder and thus it crashes. This is an issue created by the front TeraRanger sensors which are able to provide only 3 readings, and not a map showing the drone's sight of view. The trajectory of the UAV when the obstacle course is constructed with more complex geometries is presented in Appendix J.

6.2 Combined Implementation

In the future, *Potential Fields* and *D* Lite* can be combined to create an algorithm that is able

to avoid obstacles both vertically or horizontally. With further improvements to both Hardware and Software, the UAV can be brought to a technological state that is able to convert its surroundings to obstacle maps. Based on this maps, the drone will be aware of the distances towards the obstacle in both directions. By wanting to minimize the distance, there will be a switch between the algorithms allowing the UAV to select the appropriate method.

6.3 Estimation of Wind Field

As discussed previously, wind is a large factor is destabilizing the UAV for short amounts of time, creating errors that might propagate if the IMU does not perform a successful re-calibration.

With increasing wind velocity, the thrust differences between rotors increases as well, thus increasing the moment generated by opposite rotors. If the moment is not compensated by spinning the rotors that generate less thrust faster, this moment will destabilize the UAV, making it flip and crash in the end [33].

Langelaan [34] proposed a viable solution for computing the wind field with the aid of only a GPS. One can compute the values of aircraft motion with respect to the Earth (GPS values) and then compare it with predictions computed by the dynamical model employed (ROS publishers).

Combining the kinematic and dynamic models, it is possible to compute the average value of wind speed and acceleration over a discretised time interval $[k-1, k]$ by utilising the following formula:

$$\begin{bmatrix} \mathbf{w}_{k-1} \\ \dot{\mathbf{w}}_{k-1} \end{bmatrix} = \frac{1}{\Delta t} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & -\mathbf{T}^{-1} \end{bmatrix} + [\mathbf{x}_k - \mathbf{x}_{k-1} - \Delta t f_u(\mathbf{x}_{k-1}, \mathbf{u}_{k-1})] \quad (8)$$

where $\mathbf{T}$ is the direction cosine matrix, $\mathbf{I}$ is the identity matrix, $\mathbf{x}$ is the state estimation and f_u is the change in state recorded during the time interval. This is a very advanced method as it requires knowledge about turbulence modelling, but as a start, a *Calibrating Extend Kalman Filter*, as the one proposed by Lorenz [35] can be used. This filter will transform the commands sent to the drone by modelling the input velocities to account for the encountered wind field. Thus the possibility of destabilizing and crashing decreases significantly. Also, the drone will be able to follow its original planned trajectory, without having to switch to a new one due to inaccurate state estimations. De-

tailed explanation of equation (8) can be found in Appendix K.

7 Conclusion

The project, as a whole, managed to be in a large proportion a successful one as the group was not only able to implement a multitude of algorithms, but also integrate them with real pieces of Hardware. *Gazebo* provided insightful information along the way, information which helped us correct, modify and optimize the functionality of our scripts. The tests at Silwood Park confirmed what *Gazebo* has already showed: the drone is not only able to perform basic mission profiles such as takeoff, landing and hover, but also complex ones such as GPS navigation and obstacle avoidance.

Now, from a *Flight Control* perspective, the project was a success not only due to the results provided in this report, but also because the synergy created between the group members. Thus, with increased productivity, the group managed to surpass the initial expectations and developed algorithms outside the initial requirements. The gripper created by the *Hardware* team was transformed into an automatic one, by programming a switch node for the pump via Arduino. Further on, even though in the beginning it was required to land to a precise location via GPS, we modified the mission profile such that the UAV had to track the target and then perform an automatic landing using vision algorithms.

Rather than creating another another "design concept", the team managed to create a tangible object that is a few steps away from completing the complex mission it was assigned in the beginning. All of this was achieved without the high-technological equipment in the ARL and with only 4 days of outdoor testing.

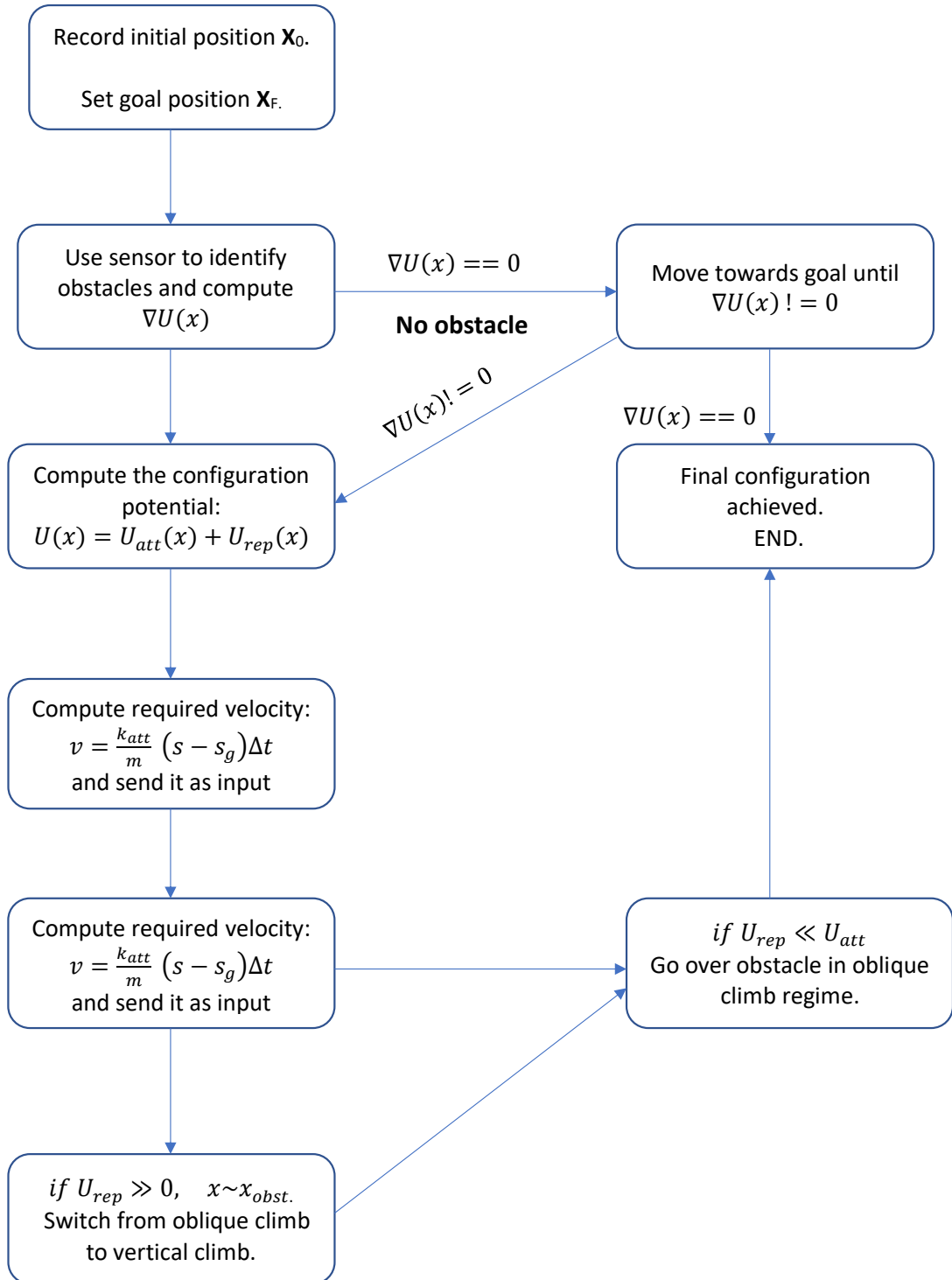
It is worth mentioning that even though Machine Learning is getting more and more progressive day by day, the interaction between a human and its surrounding environment is much more advanced than what the underlying mechanisms of a drone can emulate. Obstacles, goals or missions are concepts which we, persons, associate with different motions naturally, but an UAV has to be taught how to perceive the environment before assigning an ultimate goal; and this was the hardest task we had to accomplish. In conclusion, the project was considerably successful, as the group managed to achieve most of its predefined tasks and even more.

References

- [1] European Comission. Drones in agriculture. Internal Market, Industry, Entrepreneurship and SMEs, 2018.
- [2] Noel Sharkey. The automation and proliferation of military drones and the protection of civilians. Law, Innovation and Technology, 3(2):229–240, 2011.
- [3] Bharat Rao, Ashwin Goutham Gopi, and Romana Maione. The societal impact of commercial drones. Technology in Society, 45:83–90, 2016.
- [4] Cornelius A Thiels, Johnathon M Aho, Scott P Zietlow, and Donald H Jenkins. Use of unmanned aerial vehicles for medical product transport. Air medical journal, 34(2):104–108, 2015.
- [5] Evan Ackerman and Eliza Strickland. Medical delivery drones take flight in east africa. IEEE Spectrum, 55(1):34–35, 2018.
- [6] A. Budiyanto, A. Cahyadi, T. B. Adji, and O. Wahyunggoro. Uav obstacle avoidance using potential field under dynamic environment. In 2015 International Conference on Control, Electronics, Renewable Energy and Communications (ICCEREC), pages 187–192, Aug 2015.
- [7] Hantian Liu and M Ani Hsieh. Neural network aided information theoretic exploration. In 2018 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR), pages 1–7. IEEE, 2018.
- [8] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. In Autonomous robot vehicles, pages 396–404. Springer, 1986.
- [9] Siyue Sun, Guoqiang Yin, and Xueping Li. Path planning for mobile robot using the novel repulsive force algorithm. In IOP Conference Series: Earth and Environmental Science, volume 108, page 052067. IOP Publishing, 2018.
- [10] Rahul Kala and Kevin Warwick. Motion planning of autonomous vehicles in a non-autonomous vehicle environment without speed lanes. Engineering Applications of Artificial Intelligence, 26(5-6):1588–1601, 2013.
- [11] Soh Chin Yun, Velappa Ganapathy, and Tee Wee Chien. Enhanced d lite algorithm for mobile robot navigation. In 2010 IEEE Symposium on Industrial Electronics and Applications (ISIEA), pages 545–550. IEEE, 2010.
- [12] Pradipta Kumar Das, SN Patro, CN Panda, and Bunil Balabantaray. D* lite algorithm based path planning of mobile robot in static environment. Int. J. Comput. Commun. Technol.(IJCCT), 2:32–36, 2011.
- [13] Alexandre Borowczyk, Duc-Tien Nguyen, André Phu-Van Nguyen, Dang Quang Nguyen, David Saussié, and Jerome Le Ny. Autonomous landing of a multirotor micro air vehicle on a high velocity ground vehicle. IFAC-PapersOnLine, 50(1):10488–10494, 2017.
- [14] Jun Cheng, Yangsheng Xu, and Ronald Chung. Learning human tracking and intercepting skill. In IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004, volume 2, pages 1161–1166. IEEE, 2004.
- [15] Gazebo introduction. http://gazebo.org/tutorials?tut=guided_b1&cat=. Accessed: 2019-06-10.
- [16] Terabee/teraranger tower package. <https://github.com/Terabee/teraranger>. Accessed: 2019-06-12.
- [17] Ros/introduction. <http://wiki.ros.org/ROS/Introduction>. Accessed: 2019-06-10.
- [18] Fadri Furrer, Michael Burri, Markus Achtelik, and Roland Siegwart. Rotors—a modular gazebo mav simulator framework. In Robot Operating System (ROS), pages 595–625. Springer, 2016.
- [19] Pedro Sanchez-Cuevas, Guillermo Heredia, and Anibal Ollero. Characterization of the aerodynamic ground effect and its influence in multirotor control. International Journal of Aerospace Engineering, 2017, 2017.
- [20] Davide Del Cont Bernard, Fabio Riccardi, Mattia Giurato, and Marco Lovera. A dynamic analysis of ground effect for a quadrotor platform. IFAC-PapersOnLine, 50(1):10311–10316, 2017.

- [21] Sanjukta Aich, Chahat Ahuja, Tushar Gupta, and P Arulmozhivarman. Analysis of ground effect on multi-rotors. In 2014 International Conference on Electronics, Communication and Computational Engineering (ICECCE), pages 236–241. IEEE, 2014.
- [22] Omar D Lopez, Jaime A Escobar, and Andres M Pérez. Computational study of the wake of a quadcopter propeller in hover. In 23rd AIAA Computational Fluid Dynamics Conference, page 3961, 2017.
- [23] Juan Gonzalez-Gomez, Houxiang Zhang, and Eduardo Boemo. Locomotion principles of 1d topology pitch and pitch-yaw-connecting modular robots. In Bioinspiration and Robotics Walking and Climbing Robots. IntechOpen, 2007.
- [24] Gravity of earth.
- [25] E Gouthami and M Asha Rani. Modeling of closed loop pid controller for an auto-pilot aircraft roll control. In International Conference on recent Advances in Communication, VLSI and Embedded Systems, 2014.
- [26] Zainab Zainal, Wan Rahiman, and MNR Baharom. Yaw rate and sideslip control using pid controller for double lane changing. Journal of Telecommunication, Electronic and Computer Engineering (JTEC), 9(3-7):99–103, 2017.
- [27] Sena Temel, Semih Yağlı, and Semih Gören. P, pd, pi, pid controllers. Recitation 4 Report, Middle East Technical University, EE402-Discrete Time Control Systems, page 64, 2013.
- [28] Lin Huang, Feipeng Da, and Shaoyan Gai. Research on multi-camera calibration and point cloud correction method based on three-dimensional calibration object. Optics and Lasers in Engineering, 115:32–41, 2019.
- [29] Nancy M Amato and Yan Wu. A randomized roadmap method for path and manipulation planning. In Proceedings of IEEE international conference on robotics and automation, volume 1, pages 113–120. IEEE, 1996.
- [30] Insu Hong, Michael Kubly, and Alan T Murray. A range-restricted recharging station coverage model for drone delivery service planning. Transportation Research Part C: Emerging Technologies, 90:198–212, 2018.
- [31] Lucian Nita. Autonomous drone delivery. Technical Report - Flight Control, 2019.
- [32] MH Mabrouk and CR McInnes. Solving the potential field local minimum problem using internal agent states. Robotics and Autonomous Systems, 56(12):1050–1060, 2008.
- [33] Patricia Ventura Diaz and Steven Yoon. High-fidelity computational aerodynamics of multi-rotor unmanned aerial vehicles. In 2018 AIAA Aerospace Sciences Meeting, page 1266, 2018.
- [34] Jack W Langelaan, Nicholas Alley, and James Neidhoefer. Wind field estimation for small unmanned aerial vehicles. Journal of Guidance, Control, and Dynamics, 34(4):1016–1030, 2011.
- [35] Julian Soren Lorenz. Evaluation of different wind estimation methods in flight tests with a fixed-wing uav. 2018.
- [36] Marco A Janssen. Agent-based modelling. Modelling in ecological economics, pages 155–172, 2005.
- [37] Arend Ligtenberg, Arnold K Bregt, and Ron Van Lammeren. Multi-actor-based land use modelling: spatial planning using agents. Landscape and urban planning, 56(1-2):21–33, 2001.

A Potential Fields Method Implementation



B ROS Nodes Topology with active Topics

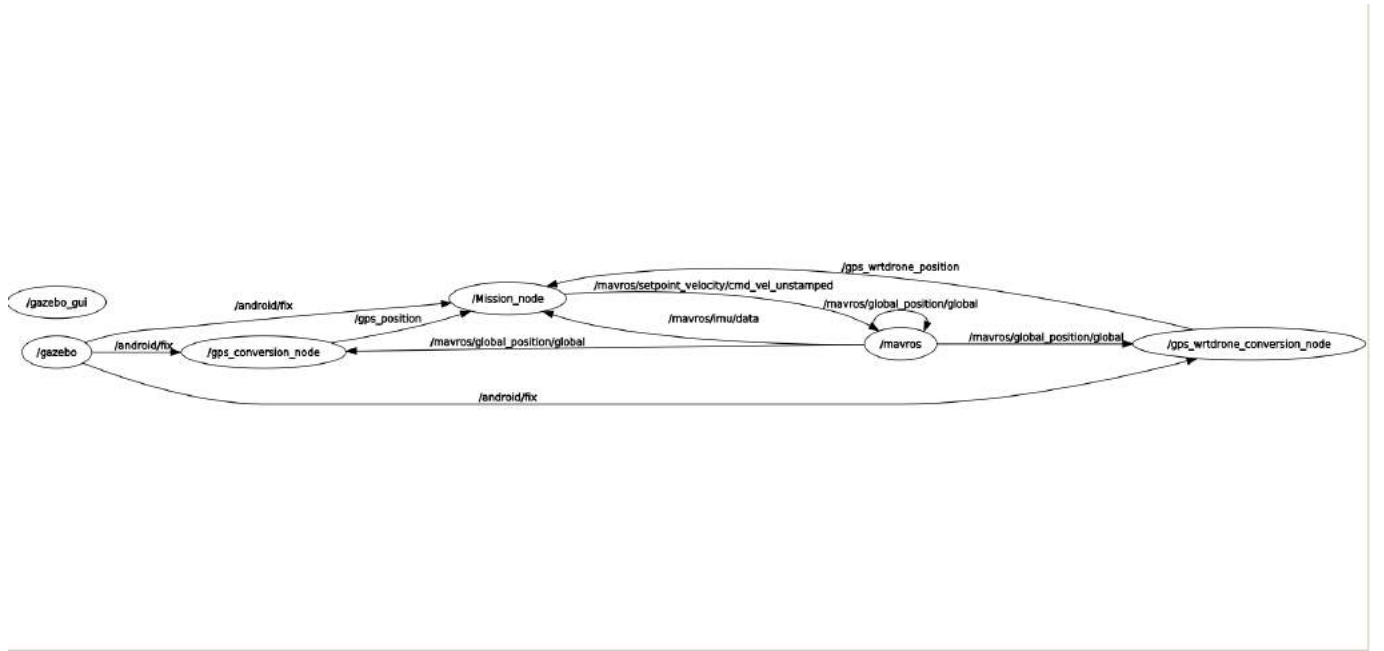


Figure 20: Full Mission ROS Node Topology

Mission_node is the master node which starts the navigation script when activated. *android/fix* and *gps_position* are the topics used to communicate the position of the ambulance and convert from Earth coordinates to local coordinates.

The *gps_wrt_position* node takes the local coordinates of both the ambulance and the UAV and computes the relative positions. The *PN Tracking* script then subscribes to these values and the whole algorithm is modelled using these values.

The navigation script is subscribed to the *IMU/data* topic published by *mavros*. With these values the state estimation is performed and thus, the drone is able to move from point A to point B while in equilibrium.

Following the dynamics and kinematics model employed in the PX4 Firmware, the global coordinates (LLA) of the drone, respectively ambulance are then sent to *Gazebo* through the topics *global_position/global* and *android/fix*. *Gazebo* then manipulates this data into visual graphics and the simulation begins.

C Obstacle Avoidance Simulations

C.1 Unit Box Test

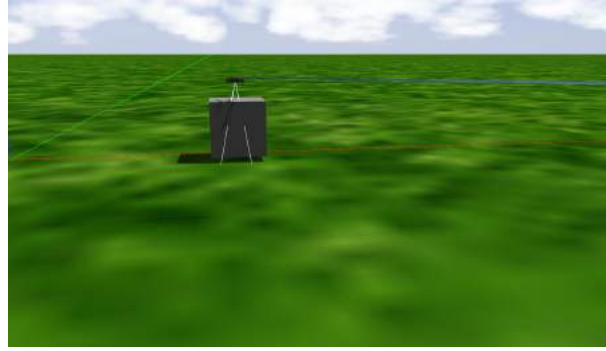
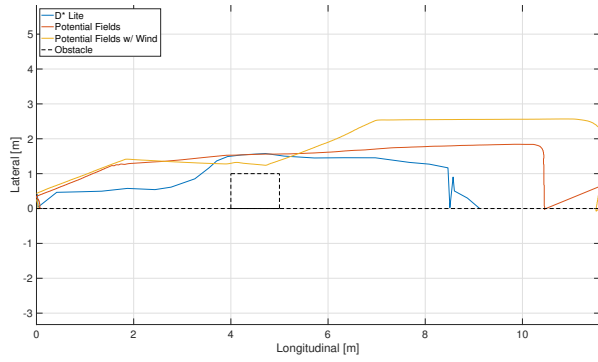
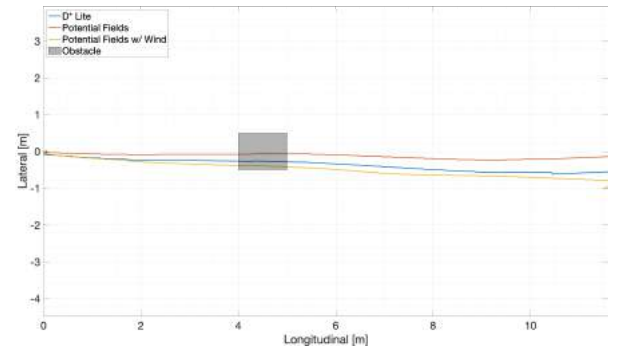


Figure 21: Unit box obstacle in Gazebo

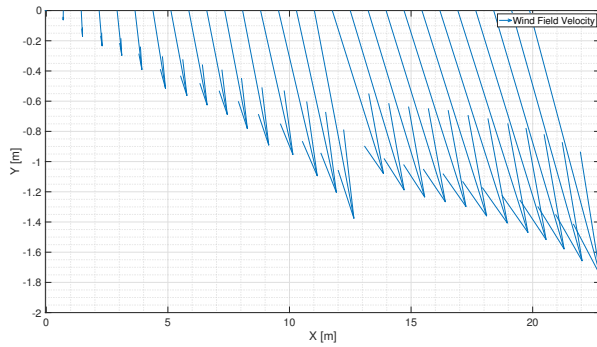


(a) Unit Box Side View

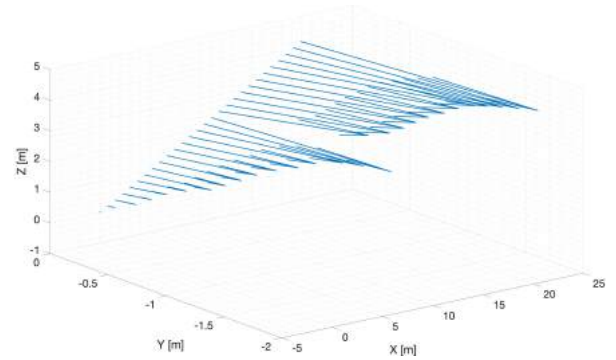


(b) Unit Box Top View

Figure 22: Trajectory Over Unit Box



(a) Wind Field - Top View



(b) Wind Field - 3D View

Figure 23: Simulated Wind Field

C.2 Unit Cylinder Test

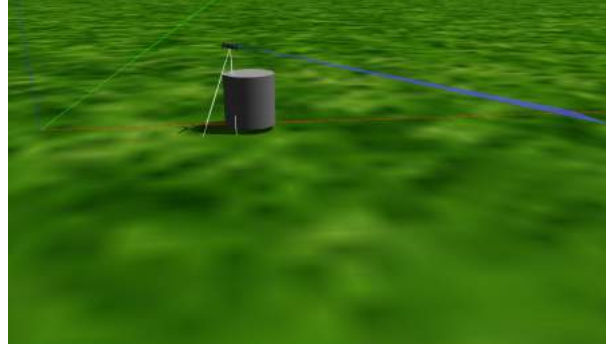
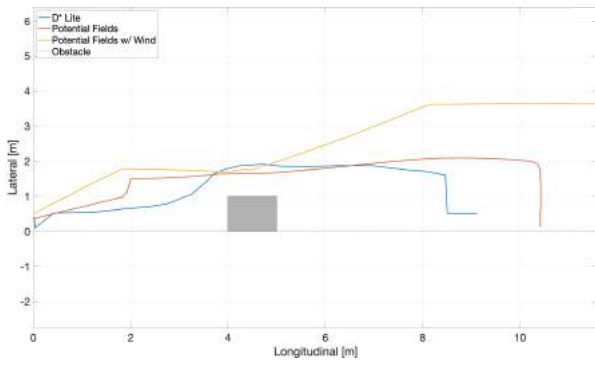
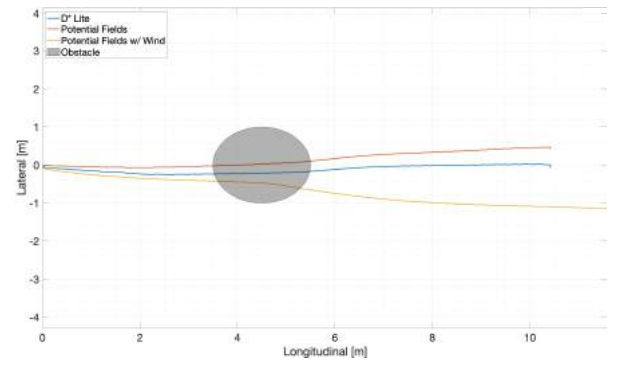


Figure 24: Unit Cylinder Obstacle in Gazebo

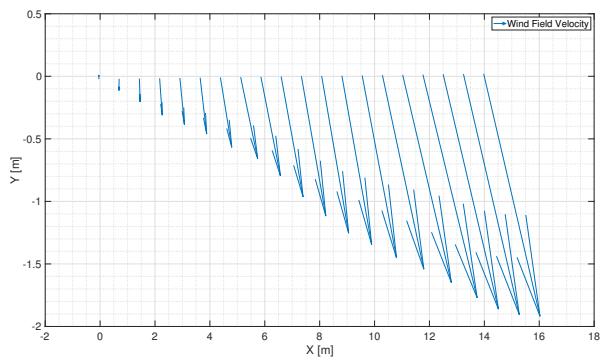


(a) Unit Cylinder Side View

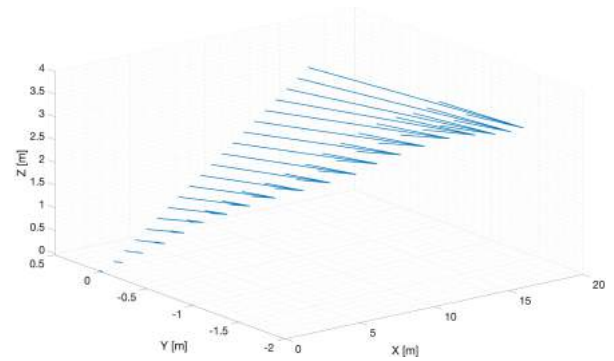


(b) Unit Cylinder Top View

Figure 25: Trajectory Over Unit Cylinder



(a) Wind Field - Top View



(b) Wind Field - 3D View

Figure 26: Simulated Wind Field

C.3 Unit Sphere Test

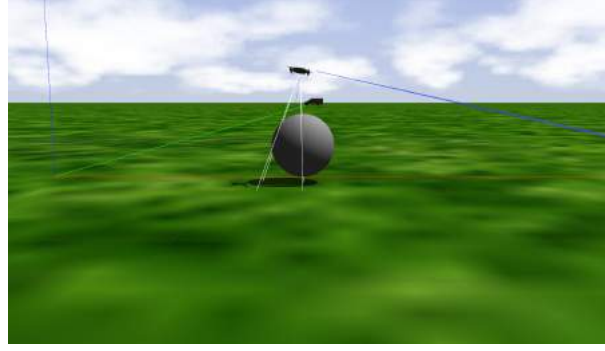
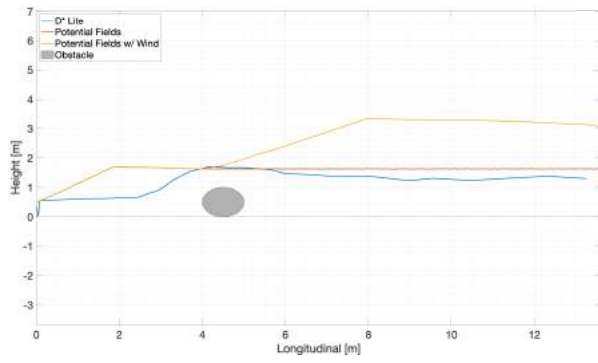
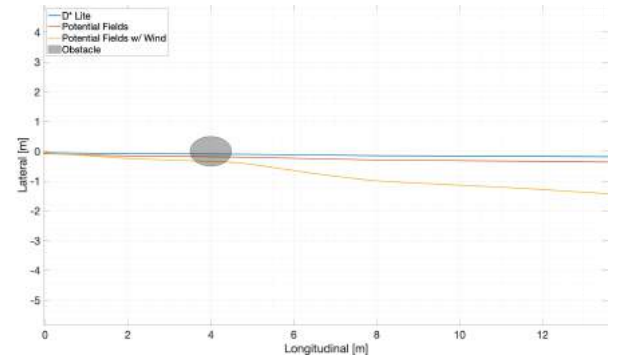


Figure 27: Unit Cylinder Obstacle in Gazebo

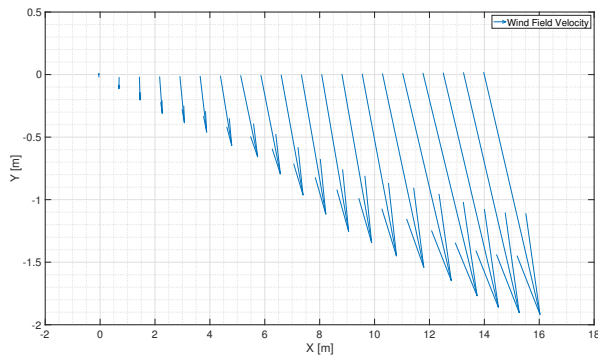


(a) Unit Sphere Side View

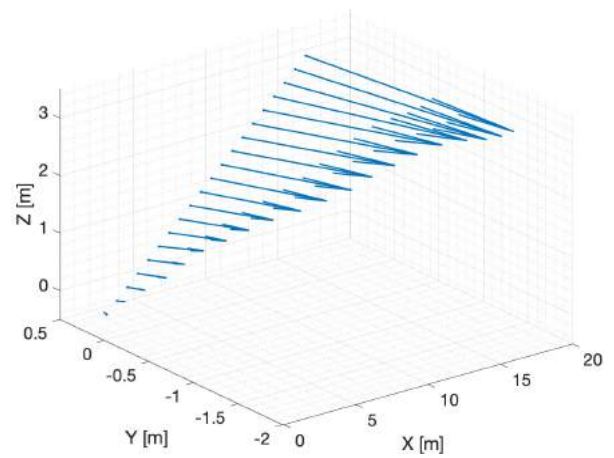


(b) Unit Sphere Top View

Figure 28: Trajectory Over Unit Sphere



(a) Wind Field - Top View



(b) Wind Field - 3D View

Figure 29: Simulated Wind Field

C.4 Unit Combined Test

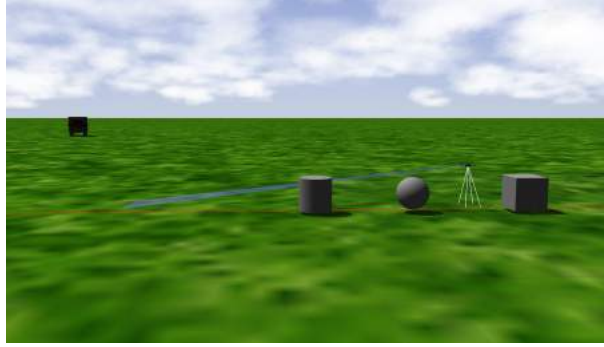
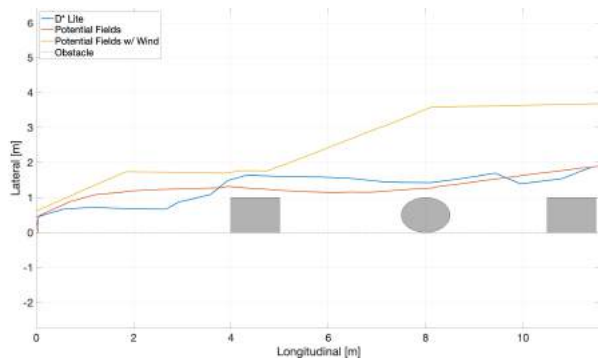
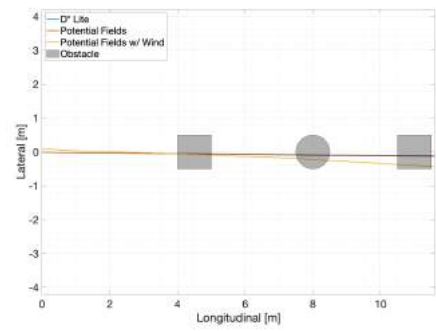


Figure 30: Unit Geometries Obstacle Course in Gazebo

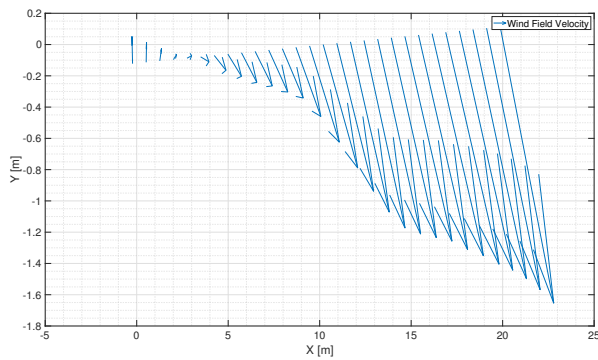


(a) Combined Geometries Side View

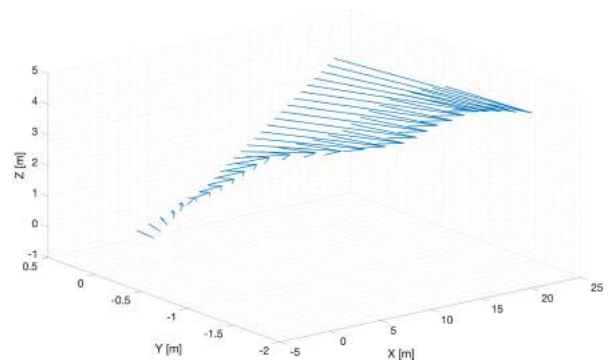


(b) Combined Geometries Top View

Figure 31: Trajectory Over Combined Unit Obstacles



(a) Wind Field - Top View



(b) Wind Field - 3D View

Figure 32: Simulated Wind Field

D CPU & Memory Usage Comparison

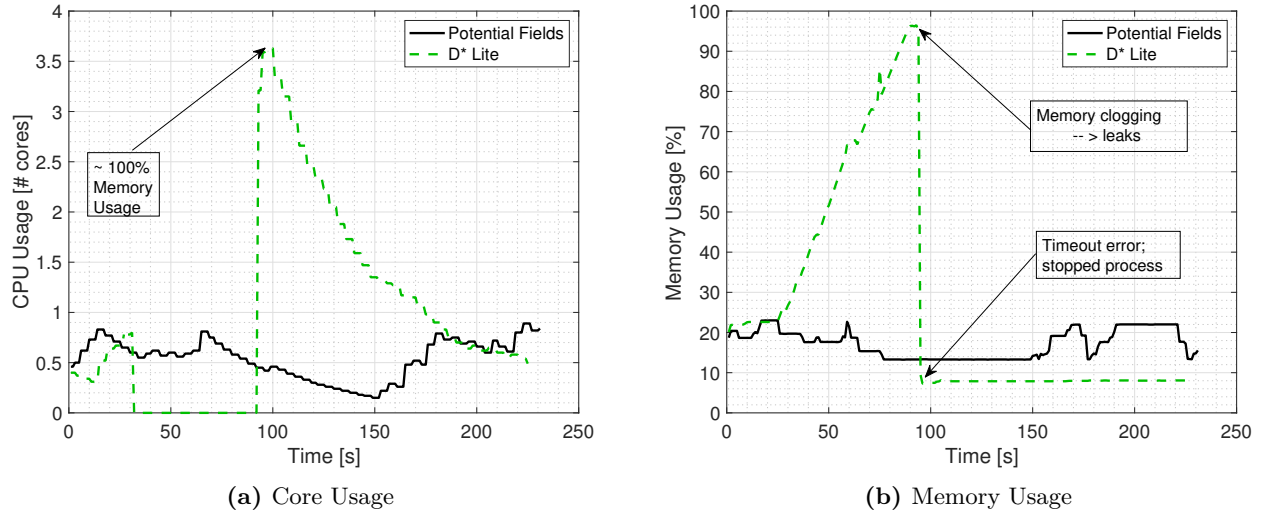


Figure 33: Computational Performance Comparison

E Numerical Comparison Methodology

The data imported to Matlab was recorded from *Gazebo* using the *rosbag* function. This function saved the data in a binary file format and the *Robotics Operating Toolbox* from Matlab has been used to read the values. The first problem that was encountered was that the arrays containing the coordinates of the drone with different algorithms implemented were not matching, thus we could not create a 100% accurate comparison. The solution? A function in Matlab was created with the goal of trimming the arrays such that their length coincided.

As mentioned in the main part of the report, the reference curve when comparing the heights at which the UAV operated was the trajectory of the drone when equipped with the Potential Fields algorithm.

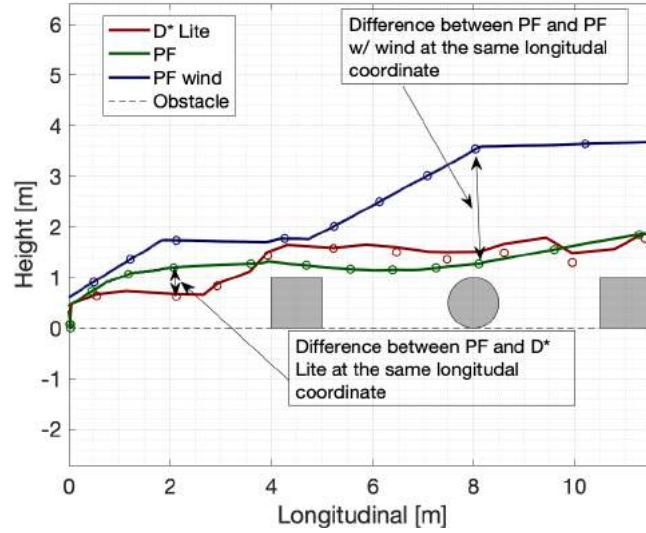


Figure 34: Height Trajectory Comparison

Figure 34 shows how the relative position values were created. After that the standard deviation and mean formulae have been applied to obtain the values presented in Tables 1 and 2.

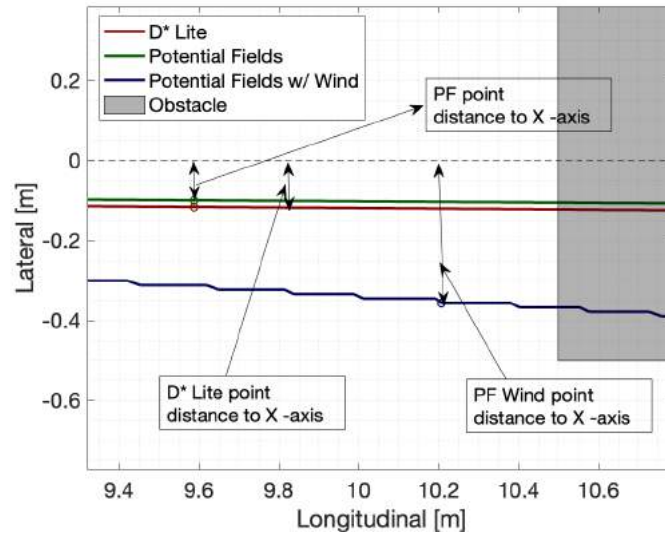


Figure 35: Lateral Trajectory Comparison

For the lateral trajectory the X-axis was chosen as the reference line, because the drone was not supposed to move sideways. It was explained in the main part of the report why the drone had an induced small angle. The same calculations were then produced for this case.

F Yaw Rate PD Controller

The same mission profile was tested in *Gazebo* before and after the implementation of the PD controller detailed in the Section 5 *Improvements*. The results are presented below:

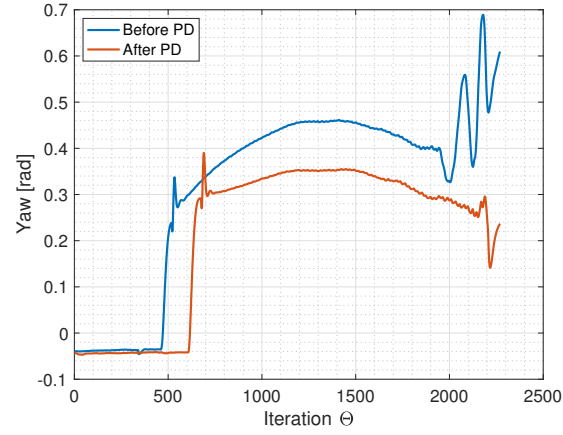
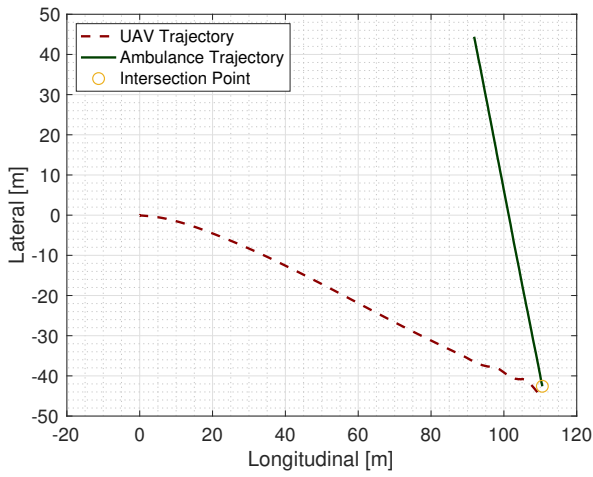
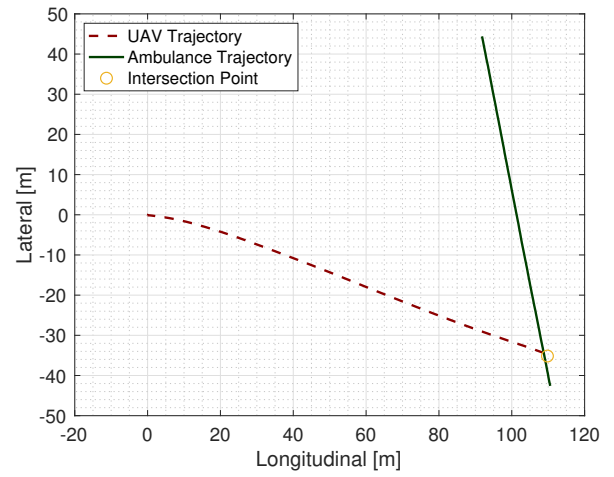


Figure 36: Yaw Comparison before/after PD Implementation



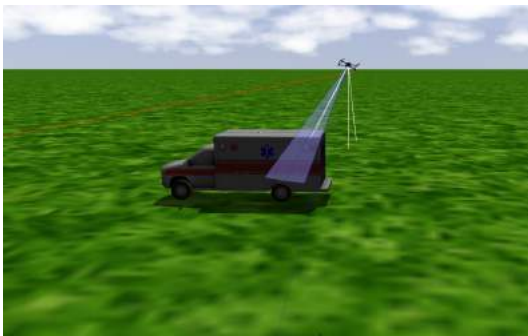
(a) 1st Tracking Trajectory



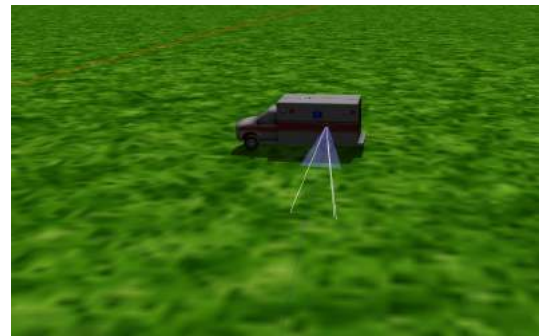
(b) 2nd Tracking Trajectory

Figure 37: Trajectory Comparison

From figure 38 it can be observed that after the implementation of the PD controller the UAV managed to track the ambulance faster while following a more fluent trajectory. In the 1st case, at around 100m longitude, the drone exhibits some jerky movements, whereas in the 2nd case it does not.



(a) Wrong orientation: drone is not facing its target



(b) Correct orientation: drone is facing its target

Figure 38: Orientation Comparison in *Gazebo*

G Obstacle Avoidance & PN Tracking Simulations

The integration of both algorithms has been simulated for more mission profiles and a couple of high-complexity ones are presented in this section:

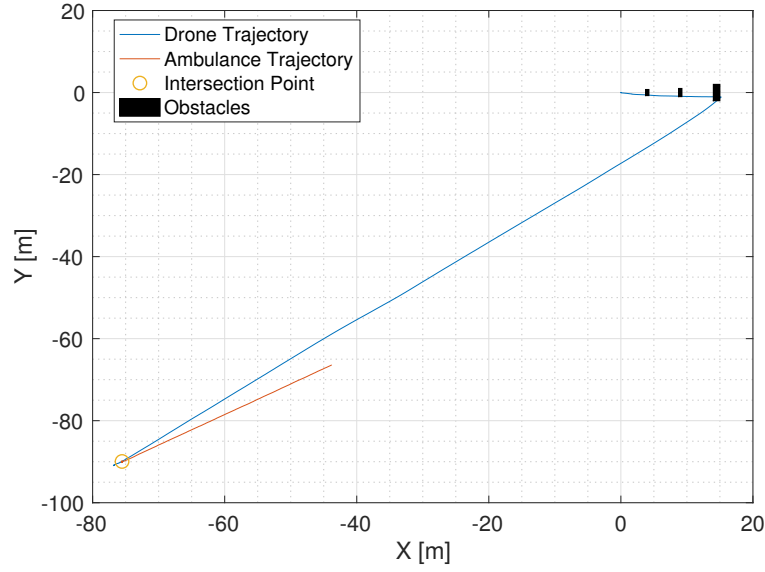


Figure 39: 1st High-Complexity Integrated Mission Profile

Figure 39 presents a case where the drone had to clear the obstacle course, then turn and track the ambulance. The UAV recognized that it has cleared the obstacle section, then performed a 135° above the last obstacle and started its quest to catch the target.

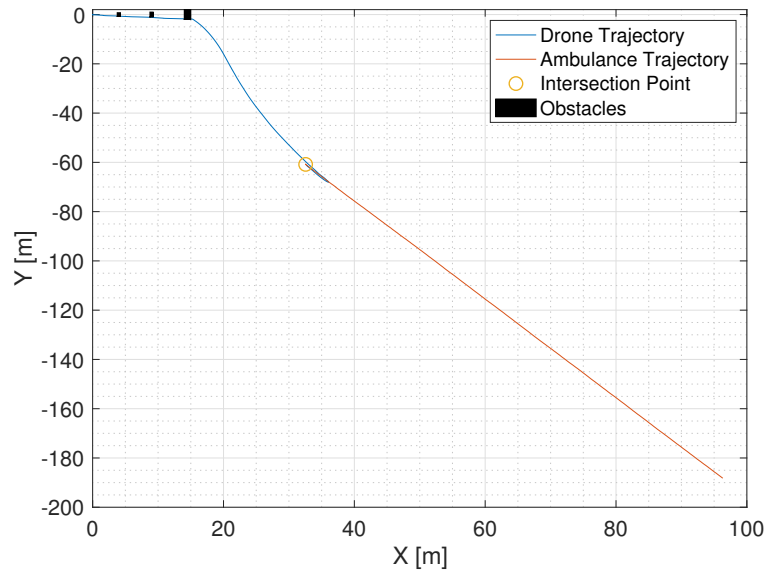


Figure 40: 2nd High-Complexity Integrated Mission Profile

Figure 40 presents the case where the ambulance's orientation was towards the UAV. The drone identify this and quickly managed to get in reaching distance to perform autonomous landing on top of the ambulance.

H Point Cloud & RViz

RViz is a 3D visualizer for displaying sensor data and state information from ROS. The *Sensors* group created a script that could model the data provided by the 3D camera, which would in the end provide a Point Cloud, allowing the UAV to identify the full shape of the obstacle. Unfortunately, they also ruled out the possibility of using this piece of Hardware due to low outdoor precision.

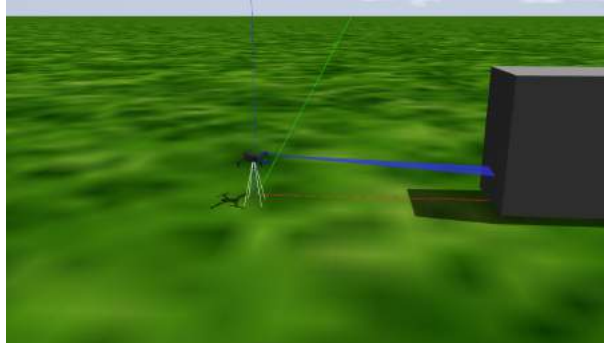
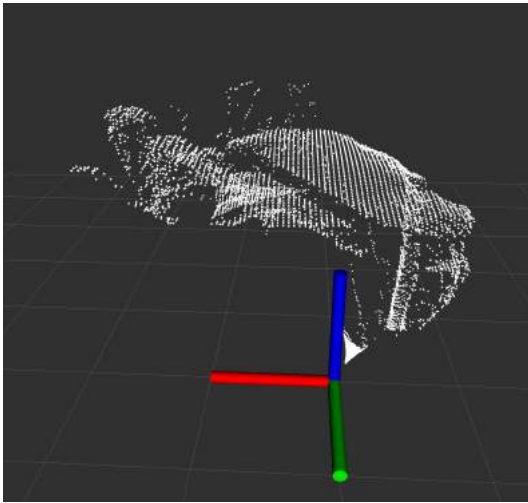
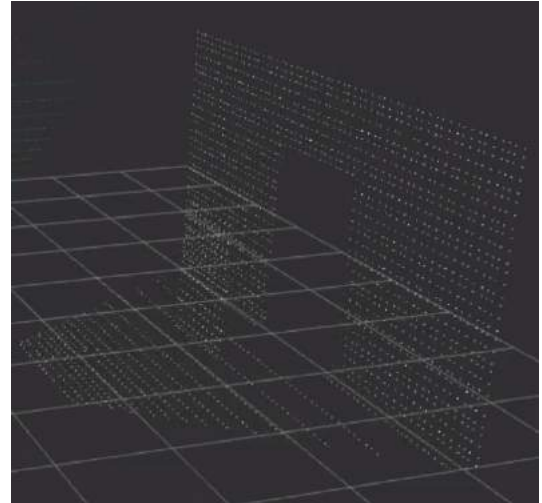


Figure 41: UAV identifying the obstacle using 3D Camera



(a) Point Cloud with no Voxel Filter



(b) Point Cloud with Voxel Filter

Figure 42: Point Cloud Comparison in *RViz*

I Dynamic Internal Agents States Solution

We begin to explain the solution by defining what an "agent" represents. According to [36], an agent can be visualized as a system that perceives its surrounding environment through sensors. Then, upon trying to reach its final goal, the agent acts upon the environment utilising its actuators.

A "swarm" [37] represents an environment where multiple agents are enforced. The interaction between them can be modelled with the aid of a 2-body Morse potential $U_{cohesion}(\mathbf{r}_i)$ which creates a field characterised by:

- *LONG* range - *WEAK* attractive force
- *SHORT* range - *STRONG* repulsive force

The Potential Field (entire free environmental space) will then be split into a subset of attractive and repulsive potential fields of strength C_{att} and C_{rep} with ranges l_{att} and l_{rep} . The obstacles and goals will be assigned potentials strength of C_{io} and C_{ig} , respectively ranges l_{io} and l_{ig} . The total potential of the free space can be computed using:

$$U_{total}(\mathbf{r}_i) = U_{cohesion}(\mathbf{r}_i) + U_{goal}(\mathbf{r}_{ig}) + U_{obstacle}(\mathbf{r}_{io}) \quad (9)$$

We will further employ the equation of motions for all N_p agents moving through the workspace. This allows us to create a 1st ODE whose solution will be equal to the velocity command need to be sent to the UAV:

$$\frac{d\mathbf{v}_i}{dt} = -\frac{\beta}{m}\mathbf{v}_i - \nabla_i U_{total}(\mathbf{r}_i) \quad (10)$$

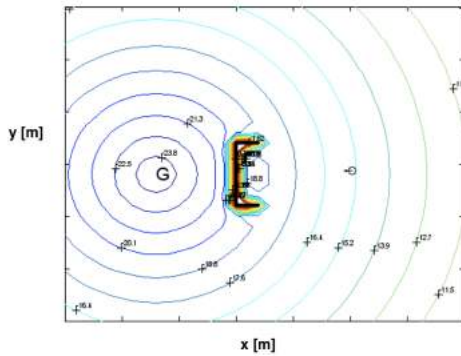
The potentials expressed in equation 9 have the following form:

$$U_{cohesion}(\mathbf{r}_i) = \sum_{j \neq i}^{N_p} \left(C_{rep_j} e^{-|\mathbf{r}_i - \mathbf{r}_j|/l_{rep_j}} - C_{att_j} e^{-|\mathbf{r}_i - \mathbf{r}_j|/l_{att_j}} \right) \quad (11)$$

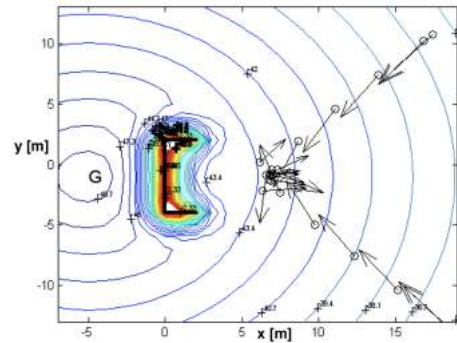
$$U_{goal}(\mathbf{r}_{ig}) = - \sum_{k=1}^{N_g} \left(C_{ig_k} e^{-|\mathbf{r}_{ig} - \mathbf{r}_{g_k}|/l_{g_k}} \right) \quad (12)$$

$$U_{obstacle}(\mathbf{r}_{io}) = \sum_{n=1}^{N_o} \left(C_{io_n} e^{-|\mathbf{r}_{io} - \mathbf{r}_{o_n}|/l_{o_n}} \right) \quad (13)$$

Now that the model of the field has been created, the agent must identify how to escape the local minima. Using higher-level perception algorithms, such as **QLearning**, the UAV will be able to differentiate the type of field is going through. Furthermore, with the aid of additional sensor information, the drone will recognize the path leading to the goal, while avoiding repulsive potentials given by the obstacles. With the potential field obtained by summing the components of equations 11, 12 and 13, the global minima points will be the only ones who have stable equilibria highlighted by a potential gradient equal to 0, $\nabla U(\mathbf{x}) = 0$. This creates the optimal trajectory for the drone, allowing it to not get stuck in local minima positions.



(a) Potential Field with Internal Agents Visualization



(b) Solved Internal "Swarm". UAV detects repulsive potential and avoids obstacle

Figure 43: Solution for Local Minima Problem expressed in figure 18

J D* Lite Lateral Simulations

As presented in the main part of this report, the implemented lateral avoidance version of the *D* Lite* Algorithm was not successful when the obstacles did not have a linear geometry. For this, a mission profile containing a sphere and a cylinder was constructed. Multiple scenarios were encountered and this requires further investigation. It is worth mentioning that the dynamic properties of the obstacles were canceled such that the drone could move them at impact.

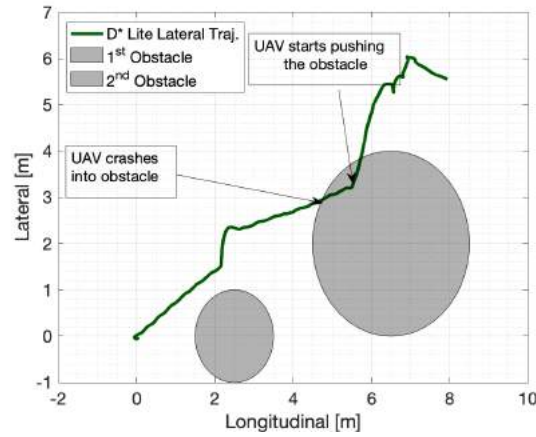


Figure 44: 1st obstacle avoided; 2nd obstacle crash

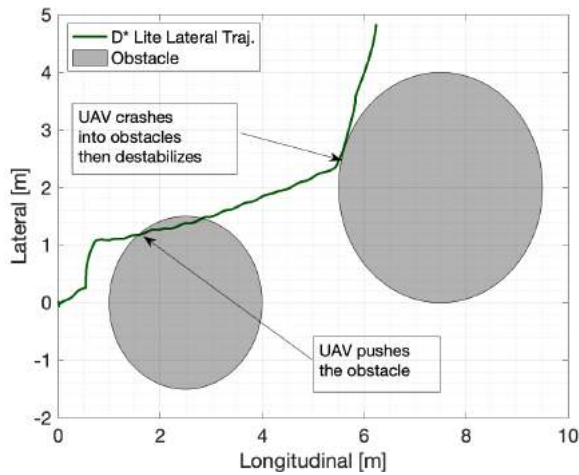


Figure 45: 1st obstacle crash; 2nd obstacle crash

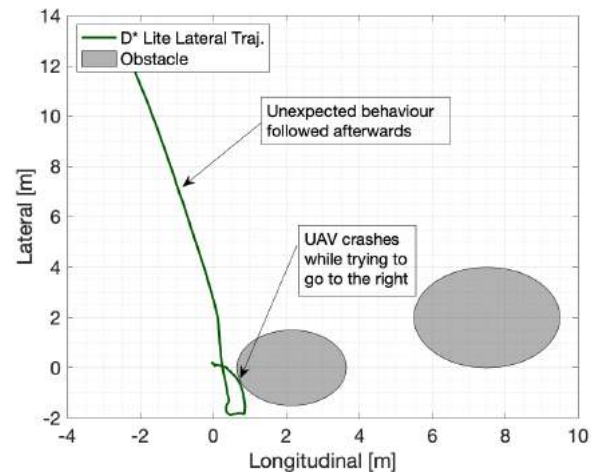


Figure 46: 1st obstacle crash then unexpected pathing

K Wind Field Estimation Equation

Starting with the kinematics equations with respect to the Earth:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \mathbf{T}^{-1} \begin{bmatrix} u \\ v \\ w \end{bmatrix} + \begin{bmatrix} w_{ix} \\ w_{iy} \\ w_{iz} \end{bmatrix} \quad (14)$$

where w_{ij} are the wind speed components in Earth FoR, $\mathbf{T}$ is the cosine matrix and $[u, v, w]$ is the velocity vector of the UAV.

Now substituting the vehicle dynamics equations in equation 14 it is obtained that:

$$\begin{bmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{bmatrix} = \mathbf{T}^{-1} \begin{bmatrix} \frac{X}{m} - g \sin \theta - q\mathbf{w} + rv \\ \frac{Y}{m} + g \sin \phi \cos \theta + p\mathbf{w} - ru \\ \frac{Z}{m} + g \cos \phi \cos \theta + qu - pv \end{bmatrix} + \mathbf{T} \begin{bmatrix} \dot{w}_{ix} \\ \dot{w}_{iy} \\ \dot{w}_{iz} \end{bmatrix} \quad (15)$$

Combining equations 14 and 15 while using forwards integration allows us to create a state equation, where the state is represented by the velocity and acceleration vectors of the wind.

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \Delta t f_u(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \Delta t \begin{bmatrix} \mathbf{w}_{k-1} \\ \dot{\mathbf{w}}_{k-1} \end{bmatrix} \quad (16)$$

The last step is to manipulate the relationship such that it allows us to retrieve the wind speed and acceleration equations in matrix form:

$$\begin{bmatrix} \mathbf{w}_{k-1} \\ \dot{\mathbf{w}}_{k-1} \end{bmatrix} = \frac{1}{\Delta t} \begin{bmatrix} I & 0 \\ 0 & -T^{-1} \end{bmatrix} + [\mathbf{x}_k - \mathbf{x}_{k-1} - \Delta t f_u(\mathbf{x}_{k-1}, \mathbf{u}_{k-1})] \quad (17)$$